МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного образовательного учреждения
высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Сопроводительная записка

по специальности: 09.02.07 Информационные системы и программирование

Руководитель
_____ /Д.Б. Берестов/
«16» марта 2026 г.

Исполнитель
_____ /М.В. Аксенов/
«16» марта 2026 г.

Пермь, 2026 г.

ОГЛАВЛЕНИЕ

Схема базы данных 3
Диаграмма классов..... 4
Подробное описание выполненных действий..... 5
Этап 1. Формирование библиотеки классов..... 5
Этап 2. Формирование приложения WPF..... 12
Этап 3. Формирование основной логики приложения в .Net Framework 25
Этап 4. Создание модульных тестов. 43
Заключение 50

Схема базы данных

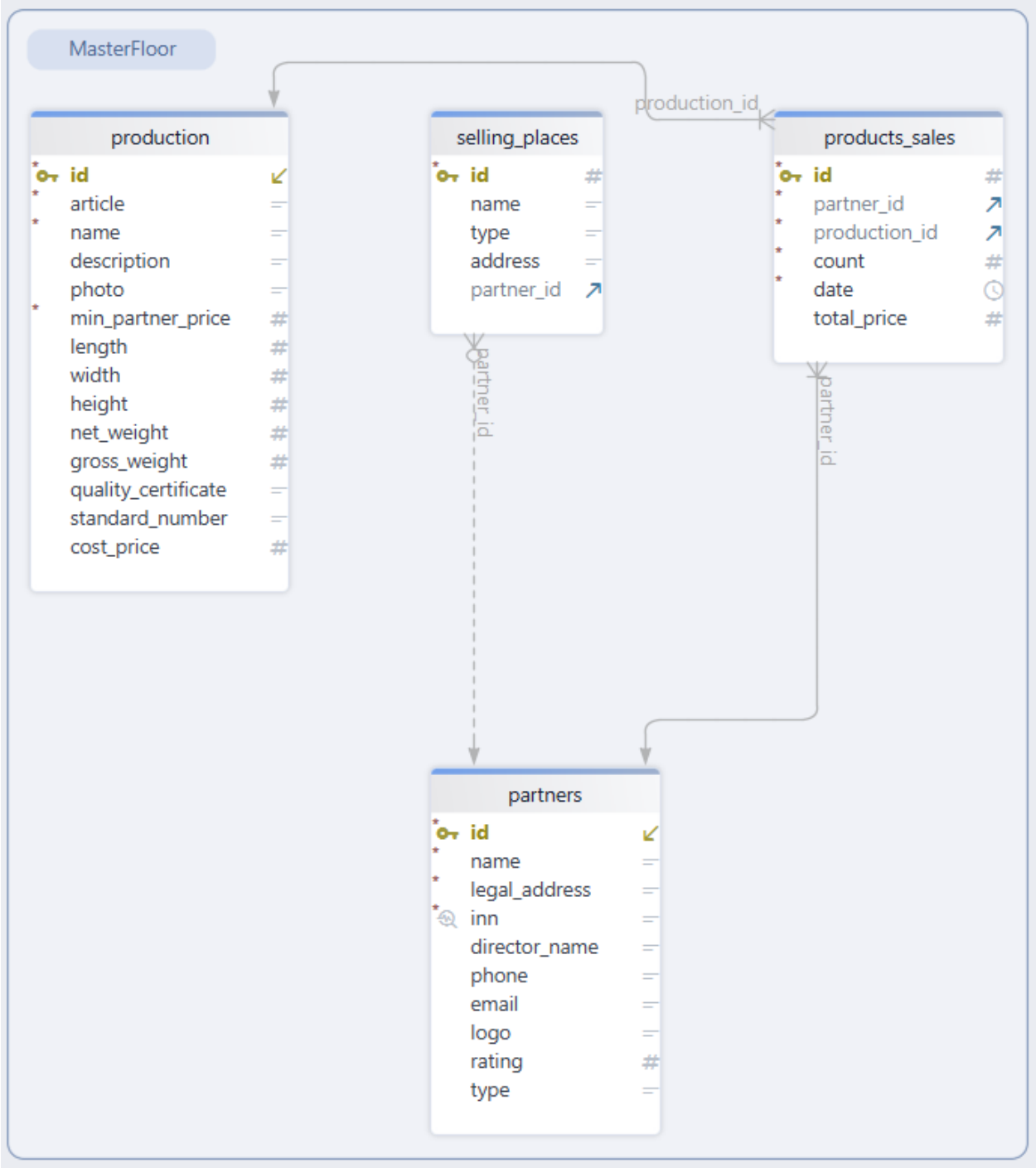
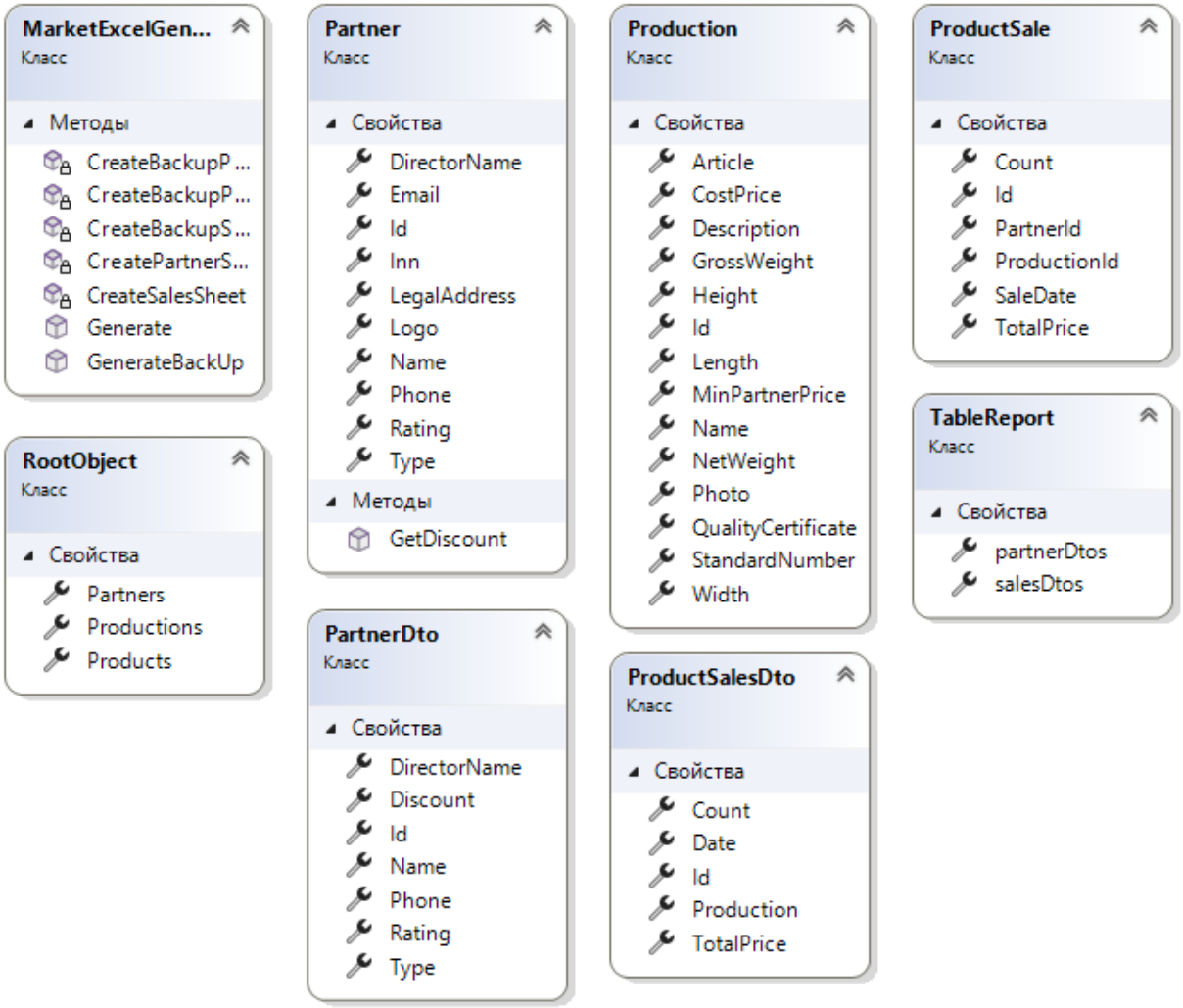


Диаграмма классов



Подробное описание выполненных действий

Этап 1. Формирование библиотеки классов

1. Используя MS Visual Studio .NET Framework, формируем класс Partner.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("partners", Schema = "app")]
    public class Partner
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [DataMember]
        [Column("id")]
        public int Id { get; set; }

        [Required]
        [DataMember]
        [Column("type")]
        public string Type { get; set; }

        [Required]
        [DataMember]
        [Column("name")]
        public string Name { get; set; }

        [Required]
        [DataMember]
        [Column("legal_address")]
        public string LegalAddress { get; set; }

        [Required]
        [DataMember]
        [Column("inn")]
        public string Inn { get; set; }  // Именование без заглавных букв, как
принято

        [DataMember]
        [Column("director_name")]
        public string DirectorName { get; set; }

        [DataMember]
        [Column("phone")]
        public string Phone { get; set; }

        [DataMember]
        [Column("email")]
        public string Email { get; set; }

        [DataMember]
        [Column("logo")]
        public string? Logo { get; set; }

        [DataMember]
        [Column("rating")]
```

```
        public int Rating { get; set; }

        public int GetDiscount(List<ProductSale> productSales)
        {
            double totalproducts = 0;
            List<ProductSale> PartnerProductSales = productSales.FindAll(x =>
x.PartnerId == Id);
            foreach (ProductSale PartnerProductSalesItem in PartnerProductSales)
            {
                totalproducts += PartnerProductSalesItem.Count;
            }
            bool Discount0 = totalproducts < 10000;
            bool Discount5 = totalproducts > 10000 && totalproducts < 50000;
            bool Discount10 = totalproducts > 50000 && totalproducts < 300000;
            bool Discount15 = totalproducts > 300000;
            if (Discount0)
                return 0;
            else if (Discount5)
                return 5;
            else if (Discount10)
                return 10;
            else
                return 15;
        }
    }
}
```

2. Формируем класс Production

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("production", Schema = "app")] // указание схемы, если нужно
    public class Production
    {
        [DataMember]
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [Column("id")]
        public int Id { get; set; }

        [DataMember]
        [Required]
        [Column("article")]
        public string Article { get; set; }

        [DataMember]
        [Required]
        [Column("name")]
        public string Name { get; set; }

        [DataMember]
        [Column("description")]
        public string Description { get; set; }

        [DataMember]
        [Column("photo")]
        public string Photo { get; set; }

        [DataMember]
```

```
        [Required]
        [Column("min_partner_price")]
        public double MinPartnerPrice { get; set; }

        [DataMember]
        [Column("length")]
        public int? Length { get; set; }

        [DataMember]
        [Column("width")]
        public int? Width { get; set; }

        [DataMember]
        [Column("height")]
        public int? Height { get; set; }

        [DataMember]
        [Column("net_weight")]
        public double? NetWeight { get; set; }

        [DataMember]
        [Column("gross_weight")]
        public double? GrossWeight { get; set; }

        [DataMember]
        [Column("quality_certificate")]
        public string QualityCertificate { get; set; }

        [DataMember]
        [Column("standard_number")]
        public string StandardNumber { get; set; }

        [DataMember]
        [Column("cost_price")]
        public double? CostPrice { get; set; }
    }
}
```

3. Формируем класс ProductSale

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Runtime.Serialization;

namespace ClassLibrary.Classes
{
    [DataContract]
    [Table("products_sales", Schema = "app")]
    public class ProductSale
    {
        [DataMember]
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        [Column("id")]
        public int Id { get; set; }

        [DataMember]
        [Column("partner_id")]
        public int PartnerId { get; set; }

        [DataMember]
        [Column("production_id")]
        public int ProductionId { get; set; }
```

```
        [DataMember]
        [Column("count")]
        public double Count { get; set; }

        [DataMember]
        [Column("date")]
        public DateOnly SaleDate { get; set; }

        [DataMember]
        [Column("total_price")]
        public double? TotalPrice { get; set; }
    }
}
```

4. Формируем класс MarketExcelGenerator

```
using System;
using System.Collections.Generic;
using OfficeOpenXml;
using ClassLibrary.DTOs;

namespace ClassLibrary.Classes
{
    public class MarketExcelGenerator
    {
        public static byte[] Generate(TableReport tableReport, int isInOneFile)
        {
            ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
            var package = new ExcelPackage();

            switch (isInOneFile)
            {
                case 0:
                case 1:
                    // Создаем таблицу для партнеров
                    CreatePartnerSheet(package, tableReport.partnerDtos);
                    // Создаем таблицу для истории продаж, если нужен
                    if (isInOneFile == 0 || isInOneFile == 2)
                    {
                        CreateSalesSheet(package, tableReport.salesDtos);
                    }
                    break;

                case 2:
                    CreatePartnerSheet(package, tableReport.partnerDtos);
                    CreateSalesSheet(package, tableReport.salesDtos);
                    break;
            }

            return package.GetAsByteArray();
        }

        // Метод для создания таблицы партнеров
        private static void CreatePartnerSheet(ExcelPackage package,
List<PartnerDto> partnerDTOs)
        {
            var worksheet = package.Workbook.Worksheets.Add("Партнеры");
            // Заголовки таблицы
            worksheet.Cells["B2"].Value = "Тип";
            worksheet.Cells["C2"].Value = "Наименование";
            worksheet.Cells["D2"].Value = "Скидка";
            worksheet.Cells["E2"].Value = "ФИО директора";
            worksheet.Cells["F2"].Value = "Телефон";
            worksheet.Cells["G2"].Value = "Рейтинг";
```

```
            int row = 3;
            foreach (var partner in partnerDTOs)
            {
                worksheet.Cells[$"B{row}"].Value = partner.Type;
                worksheet.Cells[$"C{row}"].Value = partner.Name;
                worksheet.Cells[$"D{row}"].Value = partner.Discount;
                worksheet.Cells[$"E{row}"].Value = partner.DirectorName;
                worksheet.Cells[$"F{row}"].Value = partner.Phone;
                worksheet.Cells[$"G{row}"].Value = partner.Rating;
                row++;
            }
        }

        // Метод для создания таблицы истории продаж
        private static void CreateSalesSheet(ExcelPackage package,
List<ProductSalesDto> salesDTOs)
        {
            var worksheet = package.Workbook.Worksheets.Add("История закупок");
            // Заголовки таблицы
            worksheet.Cells["B2"].Value = "Дата";
            worksheet.Cells["C2"].Value = "Количество";
            worksheet.Cells["D2"].Value = "Продукция";
            worksheet.Cells["E2"].Value = "Итоговая сумма";

            int row = 3;
            foreach (var sale in salesDTOs)
            {
                worksheet.Cells[$"B{row}"].Value = sale.Date;
                worksheet.Cells[$"C{row}"].Value = sale.Count;
                worksheet.Cells[$"D{row}"].Value = sale.Production;
                worksheet.Cells[$"E{row}"].Value = sale.TotalPrice;
                row++;
            }
        }

        public static byte[] GenerateBackUp(RootObject rootObject)
        {
            ExcelPackage.License.SetNonCommercialPersonal("Aksenov");
            var package = new ExcelPackage();

            // Создание таблицы для партнеров
            CreateBackupPartnerSheet(package, rootObject.Partners);
            // Создание таблицы для производств
            CreateBackupProductionSheet(package, rootObject.Productions);
            // Создание таблицы для продаж товаров
            CreateBackupSalesSheet(package, rootObject.Products);

            return package.GetAsByteArray();
        }

        private static void CreateBackupPartnerSheet(ExcelPackage package,
List<Partner> partners)
        {
            var sheet = package.Workbook.Worksheets.Add("Партнеры");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Тип";
            sheet.Cells["D2"].Value = "Наименование";
            sheet.Cells["E2"].Value = "Юридический адрес";
            sheet.Cells["F2"].Value = "ИНН";
            sheet.Cells["G2"].Value = "ФИО директора";
            sheet.Cells["H2"].Value = "Телефон";
            sheet.Cells["I2"].Value = "Email";
            sheet.Cells["J2"].Value = "Логотип";
            sheet.Cells["K2"].Value = "Рейтинг";
```

```
            int row = 3;
            foreach (var partner in partners)
            {
                sheet.Cells[$"B{row}"].Value = partner.Id;
                sheet.Cells[$"C{row}"].Value = partner.Type;
                sheet.Cells[$"D{row}"].Value = partner.Name;
                sheet.Cells[$"E{row}"].Value = partner.LegalAddress;
                sheet.Cells[$"F{row}"].Value = partner.Inn;
                sheet.Cells[$"G{row}"].Value = partner.DirectorName;
                sheet.Cells[$"H{row}"].Value = partner.Phone;
                sheet.Cells[$"I{row}"].Value = partner.Email;
                sheet.Cells[$"J{row}"].Value = partner.Logo;
                sheet.Cells[$"K{row}"].Value = partner.Rating;
                row++;
            }
        }

        private static void CreateBackupProductionSheet(ExcelPackage package,
List<Production> productions)
        {
            var sheet = package.Workbook.Worksheets.Add("Производство");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Артикул";
            sheet.Cells["D2"].Value = "Наименование";
            sheet.Cells["E2"].Value = "Описание";
            sheet.Cells["F2"].Value = "Фото";
            sheet.Cells["G2"].Value = "Мини цена партнера";
            sheet.Cells["H2"].Value = "Длина";
            sheet.Cells["I2"].Value = "Ширина";
            sheet.Cells["J2"].Value = "Высота";
            sheet.Cells["K2"].Value = "Вес.net";
            sheet.Cells["L2"].Value = "Вес.брутт";
            sheet.Cells["M2"].Value = "Сертификат качества";
            sheet.Cells["N2"].Value = "Стандартный номер";
            sheet.Cells["O2"].Value = "Цена себестоимости";

            int row = 3;
            foreach (var prod in productions)
            {
                sheet.Cells[$"B{row}"].Value = prod.Id;
                sheet.Cells[$"C{row}"].Value = prod.Article;
                sheet.Cells[$"D{row}"].Value = prod.Name;
                sheet.Cells[$"E{row}"].Value = prod.Description;
                sheet.Cells[$"F{row}"].Value = prod.Photo;
                sheet.Cells[$"G{row}"].Value = prod.MinPartnerPrice;
                sheet.Cells[$"H{row}"].Value = prod.Length;
                sheet.Cells[$"I{row}"].Value = prod.Width;
                sheet.Cells[$"J{row}"].Value = prod.Height;
                sheet.Cells[$"K{row}"].Value = prod.NetWeight;
                sheet.Cells[$"L{row}"].Value = prod.GrossWeight;
                sheet.Cells[$"M{row}"].Value = prod.QualityCertificate;
                sheet.Cells[$"N{row}"].Value = prod.StandardNumber;
                sheet.Cells[$"O{row}"].Value = prod.CostPrice;
                row++;
            }
        }

        private static void CreateBackupSalesSheet(ExcelPackage package,
List<ProductSale> sales)
        {
            var sheet = package.Workbook.Worksheets.Add("Продажи товаров");
            sheet.Cells["B2"].Value = "Id";
            sheet.Cells["C2"].Value = "Partner Id";
            sheet.Cells["D2"].Value = "Production Id";
            sheet.Cells["E2"].Value = "Кол-во";
```

```
            sheet.Cells["F2"].Value = "Дата";
            sheet.Cells["G2"].Value = "Общая цена";

            int row = 3;
            foreach (var sale in sales)
            {
                sheet.Cells[$"B{row}"].Value = sale.Id;
                sheet.Cells[$"C{row}"].Value = sale.PartnerId;
                sheet.Cells[$"D{row}"].Value = sale.ProductionId;
                sheet.Cells[$"E{row}"].Value = sale.Count;
                sheet.Cells[$"F{row}"].Value = sale.SaleDate.ToString(); // Можно
оставить так или применить форматирование
                sheet.Cells[$"G{row}"].Value = sale.TotalPrice;
                row++;
            }
        }
    }

    // Остальные классы данных - без изменений, их структура выглядит логичной.
    public class RootObject
    {
        public List<Partner> Partners { get; set; }
        public List<Production> Productions { get; set; }
        public List<ProductSale> Products { get; set; }
    }

    public class TableReport
    {
        public List<PartnerDto> partnerDtos { get; set; }
        public List<ProductSalesDto> salesDtos { get; set; }
    }
}
```

5. Формируем класс PartnerDto

```
namespace ClassLibrary.DTOs
{
    public class PartnerDto
    {
        public int Id { get; set; }
        public string Type { get; set; }
        public string Name { get; set; }
        public int Discount { get; set; }
        public string DirectorName { get; set; }
        public string Phone { get; set; }
        public int Rating { get; set; }
    }
}
```

6. Формируем класс ProductSalesDto

```
using System;

namespace ClassLibrary.DTOs
{
    public class ProductSalesDto
    {
        public int Id { get; set; }
        public string Date { get; set; }
        public double Count { get; set; }
        public string Production { get; set; }
        public double TotalPrice { get; set; }
    }
}
```

Этап 2. Формирование приложения WPF.

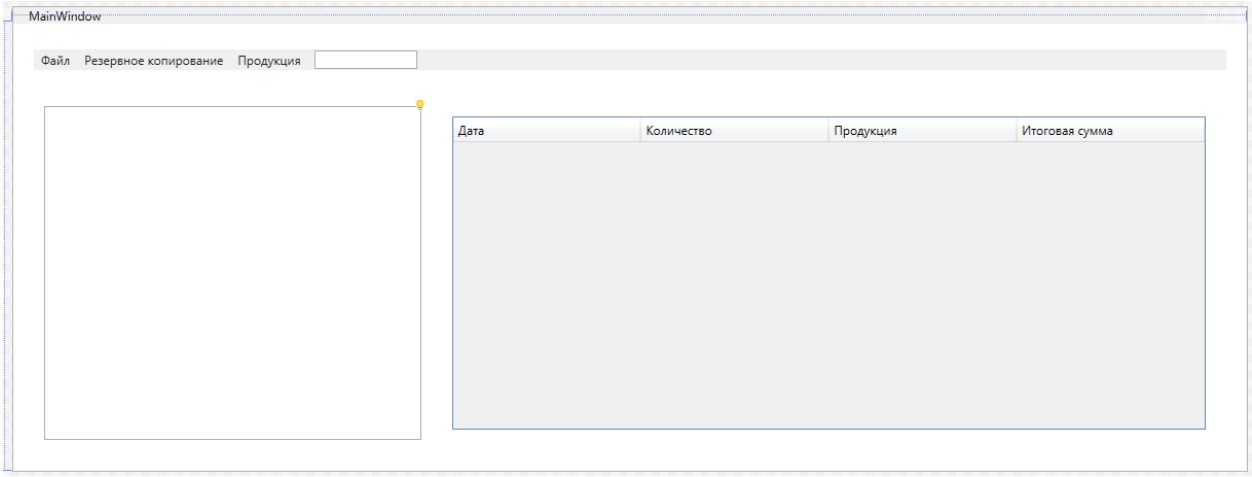
1. Приступаем к формированию главного интерфейса WPF.

```
<Window x:Class="MasterFloor.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="1200">
    <Grid>
        <Grid Margin="10">
            <Grid.RowDefinitions>
                <RowDefinition Height="50"/>
                <RowDefinition Height="*"/>
            </Grid.RowDefinitions>
            <Grid Margin="10" Grid.Row="0">
                <Menu x:Name="Menu" VerticalAlignment="Center">
                    <MenuItem Header="Файл">
                        <MenuItem Header="Сохранить Excel">
                            <MenuItem Header="Все сразу" Click="ToExcel"
CommandParameter="0"/>
                            <MenuItem Header="Партнеры" Click="ToExcel"
CommandParameter="1"/>
                            <MenuItem Header="История покупок" Click="ToExcel"
CommandParameter="2"/>
                        </MenuItem>
                    </MenuItem>
                    <MenuItem Header="Резервное копирование">
                        <MenuItem Header="Сохранить">
                            <MenuItem Header="BIN" Click="Save"/>
                            <MenuItem Header="Excel" Click="ExcelSave"/>
                        </MenuItem>
                        <MenuItem Header="Загрузить">
                            <MenuItem Header="BIN" Click="Load"/>
                            <MenuItem Header="Excel" Click="ExcelLoad"/>
                        </MenuItem>
                    </MenuItem>
                    <MenuItem Header="Продукция">
                        <MenuItem Header="Добавить" Click="AddProduction"/>
                        <MenuItem Header="Изменить" Click="RedactProduction"/>
                        <MenuItem Header="Удалить" Click="DeleteProduction"/>
                    </MenuItem>
                    <TextBox x:Name="Source_TB" Width="100"
HorizontalAlignment="Left" VerticalAlignment="Center"
TextChanged="Source_TB_TextChanged"/>
                </Menu>
            </Grid>
            <Grid Margin="10" Grid.Row="1">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" MinWidth="300"/>
                    <ColumnDefinition Width="2*" MaxWidth="800"/>
                </Grid.ColumnDefinitions>
                <Grid Margin="10, 10, 10, 10" Grid.Column="0">
                    <ListBox x:Name="Partners_LB" Padding="10"
SelectionChanged="Partners_LB_Selected">
                        <ListBox.ItemTemplate>
                            <DataTemplate>
                                <Border Padding="5, 5, 35, 5" Height="110"
VerticalAlignment="Center"
```

```
                                        BorderBrush="Black" BorderThickness="0.5"
HorizontalAlignment="Stretch"
                                        Width="{Binding ActualWidth,
RelativeSource={RelativeSource AncestorType=ListBox}}">
                                    <StackPanel Margin="5"
HorizontalAlignment="Stretch">
                                        <Grid Margin="5"
HorizontalAlignment="Stretch">
                                            <StackPanel HorizontalAlignment="Left"
Orientation="Horizontal">
                                                <TextBlock FontSize="14" Padding="5,
5, 0, 5" Text="{Binding Path=Type}" HorizontalAlignment="Left"
VerticalAlignment="Top"/>
                                                <Label Content="|"/>
                                                <TextBlock FontSize="14" Padding="0,
5, 5, 5" Text="{Binding Path=Name}" HorizontalAlignment="Left"
VerticalAlignment="Top"/>
                                            </StackPanel>
                                            <TextBlock FontSize="12" Padding="5"
Text="{Binding Path=Discount, StringFormat={}{0} %}" HorizontalAlignment="Right"
VerticalAlignment="Top"/>
                                        </Grid>
                                        <StackPanel Orientation="Vertical"
Margin="5" HorizontalAlignment="Stretch">
                                            <TextBlock FontSize="12" Text="{Binding
Path=DirectorName, StringFormat={}Директор {0}}" HorizontalAlignment="Left" />
                                            <TextBlock FontSize="12" Text="{Binding
Path=Phone}" HorizontalAlignment="Left" />
                                            <TextBlock FontSize="12" Text="{Binding
Path=Rating, StringFormat={}Рейтинг: {0}}" HorizontalAlignment="Left" />
                                        </StackPanel>
                                    </StackPanel>
                                </Border>
                            </DataTemplate>
                        </ListBox.ItemTemplate>
                        <ListBox.ContextMenu>
                            <ContextMenu>
                                <MenuItem Header="Добавить" Click="AddPartner"/>
                                <MenuItem Header="Редактировать"
Click="RedactPartner"/>
                                <MenuItem Header="Удалить" Click="DeletePartner"/>
                                <MenuItem Header="Добавить заказ"
Click="AddProductSaleSelected"/>
                            </ContextMenu>
                        </ListBox.ContextMenu>
                    </ListBox>
                </Grid>
                <Grid Margin="10" Grid.Column="1">
                    <DataGrid x:Name="ProductSales_Table" CanUserSortColumns="True"
Margin="10, 10, 10, 10" AutoGenerateColumns="False" CanUserAddRows="False"
CanUserDeleteRows="False" CanUserReorderColumns="False" IsReadOnly="True"
RowBackground="Azure" AlternatingRowBackground="White">
                        <DataGrid.Columns>
                            <DataGridTextColumn Width="*" Header="Дата"
Binding="{Binding Path=Date}"/>
                            <DataGridTextColumn Width="*" Header="Количество"
Binding="{Binding Path=Count}"/>
                            <DataGridTextColumn Width="*" Header="Продукция"
Binding="{Binding Path=Production}"/>
                            <DataGridTextColumn Width="*" Header="Итоговая сумма"
Binding="{Binding Path=TotalPrice, StringFormat={}{0} ₽}"/>
                        </DataGrid.Columns>
                        <DataGrid.ContextMenu>
                            <ContextMenu>
                                <MenuItem Header="Добавить" Click="AddProductSale"/>
```

```
                                <MenuItem Header="Редактировать"
Click="RedactProductSale"/>
                                <MenuItem Header="Удалить"
Click="DeleteProductSale"/>
                            </ContextMenu>
                        </DataGrid.ContextMenu>
                    </DataGrid>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

В результате получаем интерфейс главного окна:



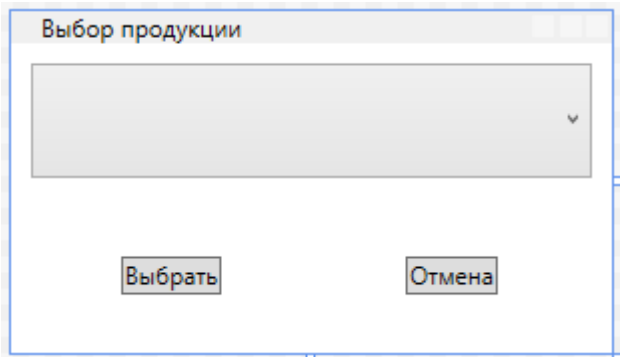
2. Далее приступаем к созданию диалоговых окон.

a. ChoiceWindow

```
<Window x:Class="MasterFloor.DataForms.ChoiceWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="Выбор продукции" MinHeight="100" MinWidth="200" MaxHeight="500"
MaxWidth="700" Height="170" Width="300">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="*"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>
        <Grid Margin="10, 10, 10, 10" Grid.Row="0">
            <ComboBox x:Name="Choice_CB"/>
        </Grid>
        <Grid Margin="10, 10, 10, 10" Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <Button x:Name="accept_btn" Content="Выбрать"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"
Grid.Column="0"/>
            <Button IsCancel="True" x:Name="reject_btn" Content="Отмена"
HorizontalAlignment="Center" VerticalAlignment="Center" Grid.Column="1"/>
```

```
        </Grid>
    </Grid>
</Window>
```

В результате получаем интерфейс окна выбора продукции:



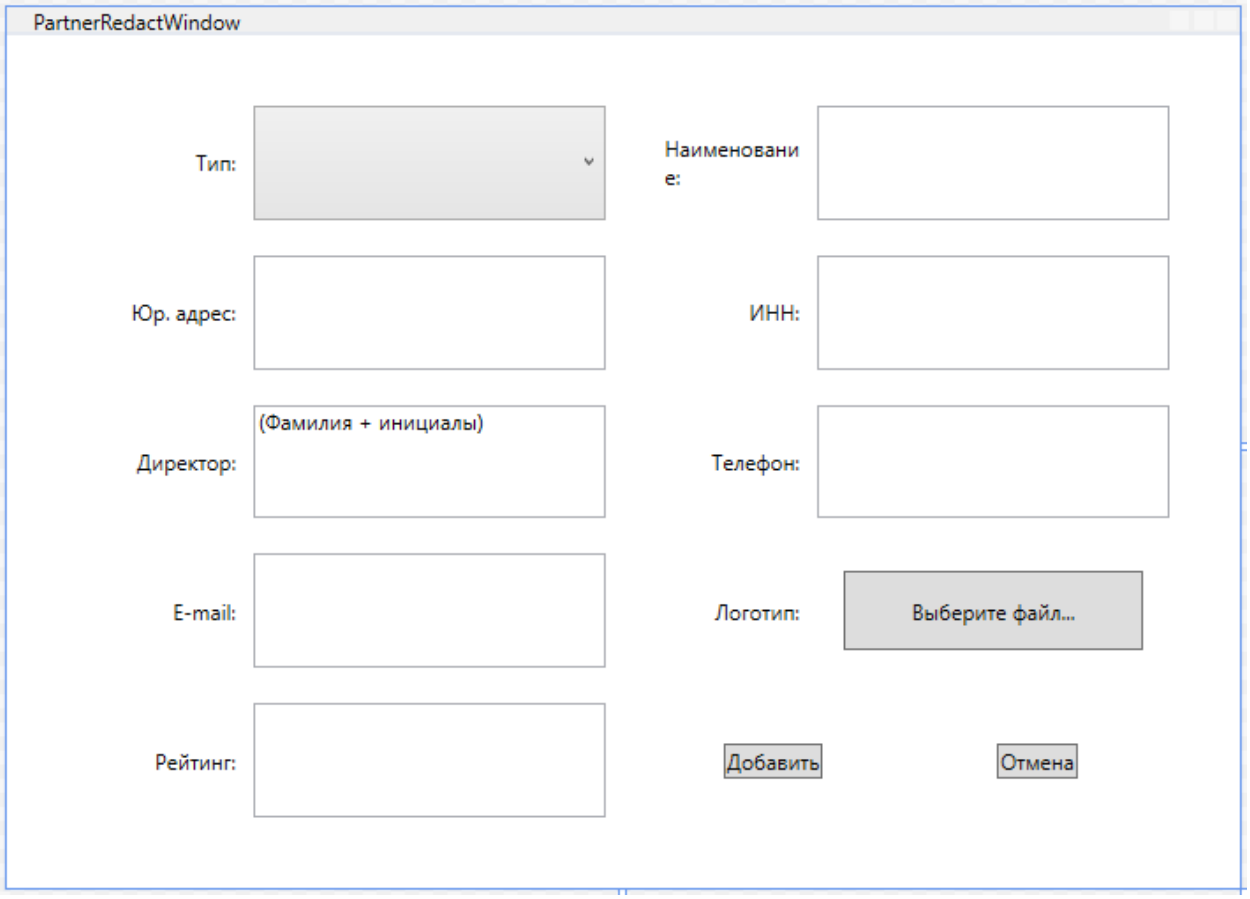
b. PartnerRedactWindow

```
<Window x:Class="MasterFloor.DataForms.PartnerRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="PartnerRedactWindow" Height="500" Width="700">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
            <Grid Margin="10, 10, 10, 10">
                <Grid Margin="10" Grid.Row="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    <Grid.RowDefinitions>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                        <RowDefinition Height="*"/>
                    </Grid.RowDefinitions>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Тип:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <ComboBox x:Name="Partner_Type_CB" Grid.Column="1">
                            <ComboBoxItem Content="ИП"/>
                            <ComboBoxItem Content="КФЛ"/>
                            <ComboBoxItem Content="ХТ"/>
                            <ComboBoxItem Content="ХО"/>
                            <ComboBoxItem Content="ООО"/>
                            <ComboBoxItem Content="АО/НАО"/>
                            <ComboBoxItem Content="ПАО"/>
                            <ComboBoxItem Content="ХП"/>
                            <ComboBoxItem Content="УП"/>
                        </ComboBox>
                    </Grid>
```

```
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10"
Text="Наименование:" HorizontalAlignment="Right" VerticalAlignment="Center"
Grid.Column="0"/>
                        <TextBox x:Name="Partner_Name_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Юр. адрес:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Legal_Address_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="ИНН:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_INN_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Директор:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Director_Name_TB" Text="(Фамилия +
инициалы)" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Телефон:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Phone_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="E-mail:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Email_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Логотип:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
```

```
                        <Button x:Name="Partner_Logo_B"
Click="Partner_Logo_Choosing" Margin="15, 10" Content="Выберите файл..."
Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="0">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="2*"/>
                        </Grid.ColumnDefinitions>
                        <TextBlock TextWrapping="Wrap" Margin="10" Text="Рейтинг:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                        <TextBox x:Name="Partner_Rating_TB" Grid.Column="1"/>
                    </Grid>
                    <Grid Margin="10, 10, 10, 10" Grid.Row="4" Grid.Column="1">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*"/>
                            <ColumnDefinition Width="*"/>
                        </Grid.ColumnDefinitions>
                        <Button x:Name="accept_btn" Content="Добавить"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"
Grid.Column="0"/>
                        <Button x:Name="reject_btn" Content="Отмена"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"
Grid.Column="1"/>
                    </Grid>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

В результате получаем интерфейс окна изменения данных о партнерах:



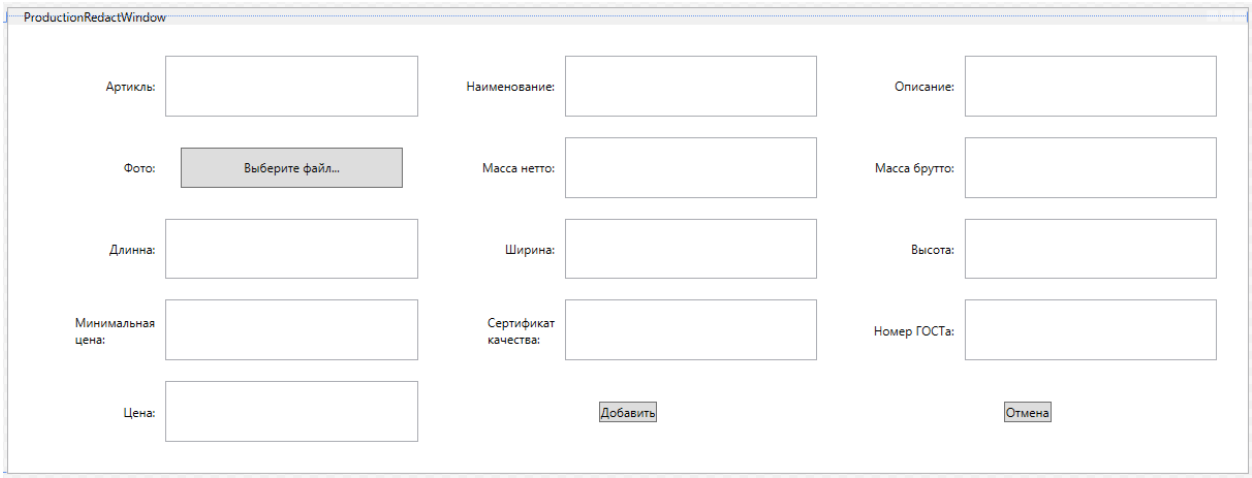
c. ProductionRedactWindow

```
<Window x:Class="MasterFloor.DataForms.ProductionRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="ProductionRedactWindow" Height="450" Width="1200">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
            <Grid Margin="10, 10, 10, 10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="*"/>
                </Grid.ColumnDefinitions>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Артикль:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Article_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Наименование:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Name_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Описание:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Description_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Фото:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <Button x:Name="Production_Photo_B"
Click="Production_Logo_Choosing" Margin="15, 10" Content="Выберите файл..."
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="1">
```

```
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Масса нетто:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Net_Weight_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Масса брутто:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Gross_Weight_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Длинна:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Length_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Ширина:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Width_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2" Grid.Column="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Высота:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Height_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Минимальная
цена:" HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Min_Partner_Price_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Сертификат
качества:" HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Quality_Certificate_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3" Grid.Column="2">
```

```
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Номер ГОСТа:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Standard_Number_TB"
Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Цена:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="Production_Cost_Price_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="1">
                    <Button x:Name="accept_btn" Content="Добавить"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="4" Grid.Column="2">
                    <Button x:Name="reject_btn" Content="Отмена"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"/>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

В результате получаем интерфейс окна изменения данных о продукции:

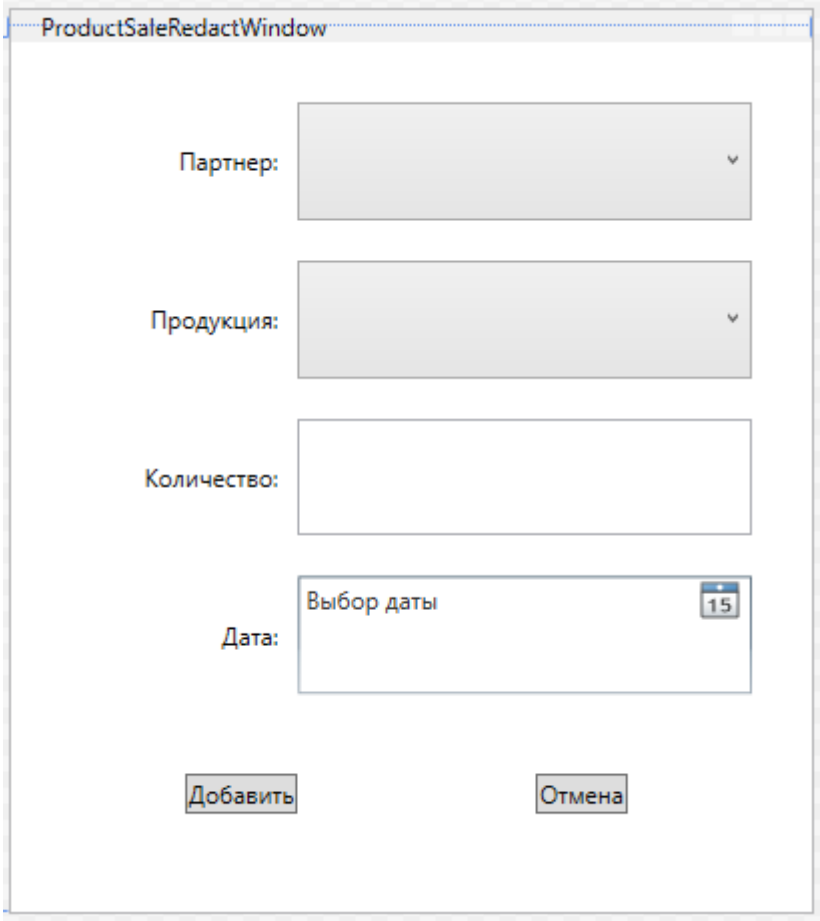


d. ProductSaleRedactWindow

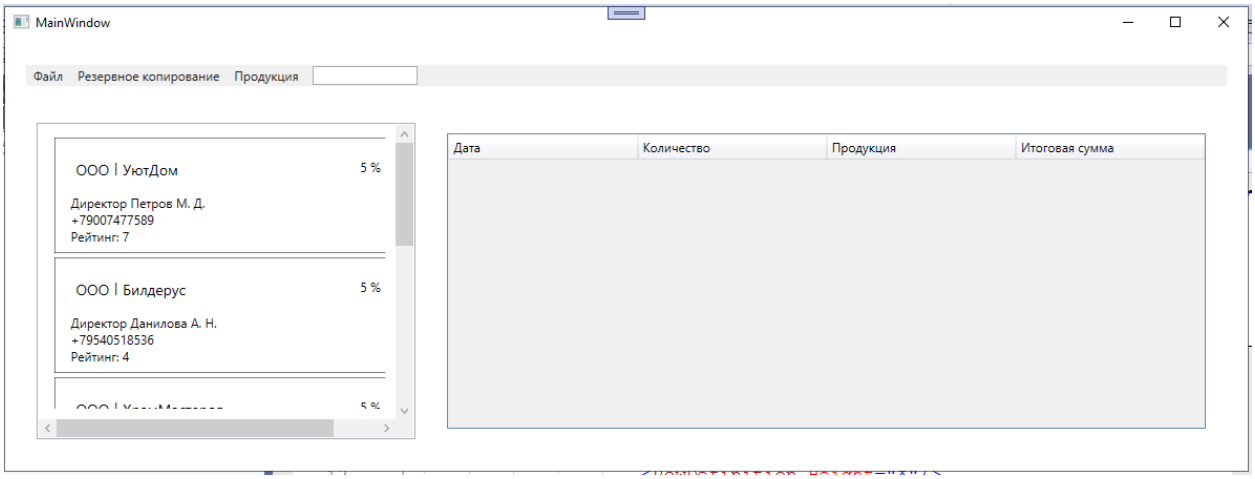
```
<Window x:Class="MasterFloor.DataForms.ProductSaleRedactWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:MasterFloor.DataForms"
        mc:Ignorable="d"
        Title="ProductSaleRedactWindow" Height="450" Width="400">
    <Grid>
        <Grid Margin="10, 10, 10, 10">
```

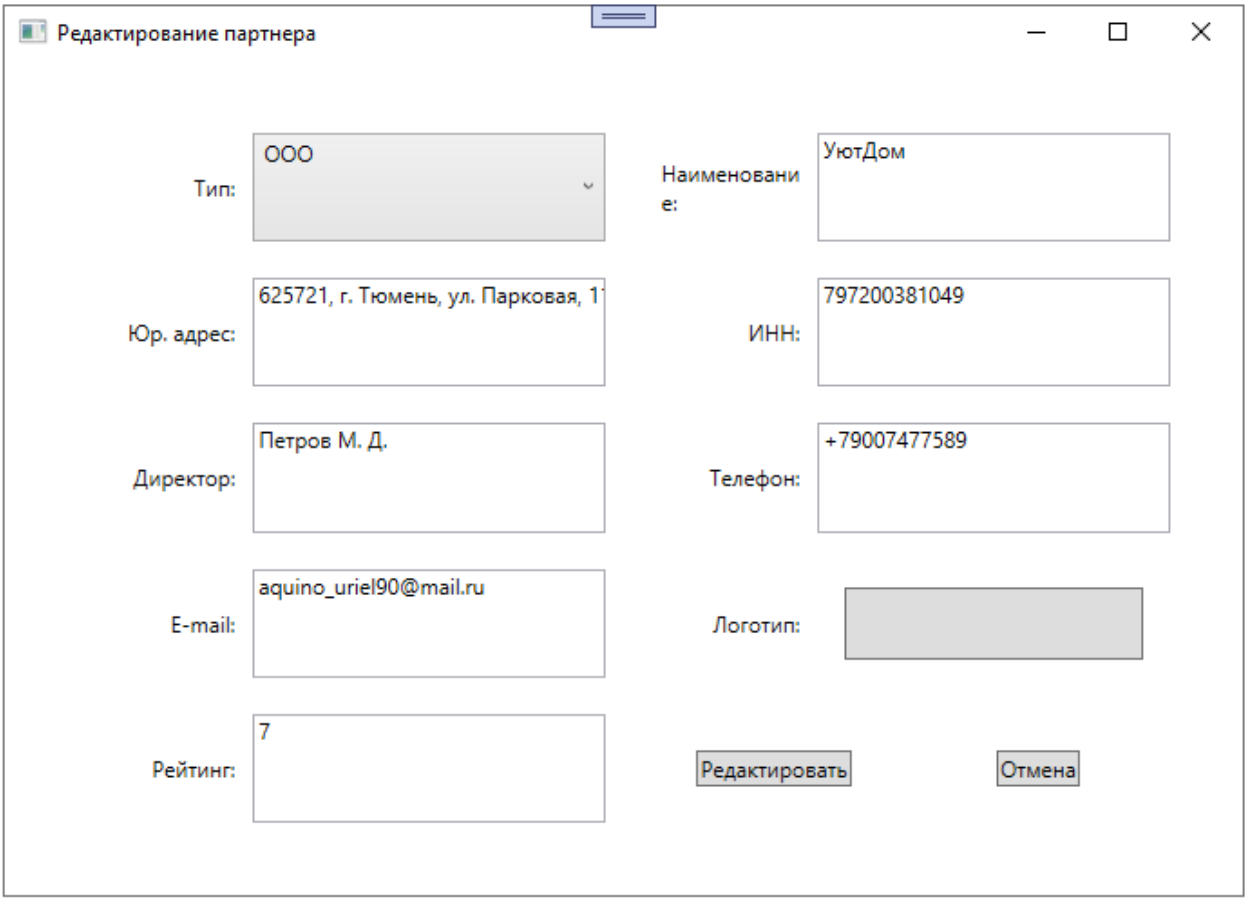
```
            <Grid Margin="10, 10, 10, 10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                    <RowDefinition Height="*"/>
                </Grid.RowDefinitions>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="0">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Партнер:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <ComboBox x:Name="ProductSale_Partner_CB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="1">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Продукция:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <ComboBox x:Name="ProductSale_Production_CB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="2">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Количество:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <TextBox x:Name="ProductSale_Count_TB" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10"  Grid.Row="3">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="2*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock TextWrapping="Wrap" Margin="10" Text="Дата:"
HorizontalAlignment="Right" VerticalAlignment="Center" Grid.Column="0"/>
                    <DatePicker x:Name="ProductSale_Date_DP" Grid.Column="1"/>
                </Grid>
                <Grid Margin="10, 10, 10, 10" Grid.Row="4">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    <Button x:Name="accept_btn" Content="Добавить"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="accept_btn_Click"
Grid.Column="0"/>
                    <Button x:Name="reject_btn" Content="Отмена"
HorizontalAlignment="Center" VerticalAlignment="Center" Click="reject_btn_Click"
Grid.Column="1"/>
                </Grid>
            </Grid>
        </Grid>
    </Grid>
</Window>
```

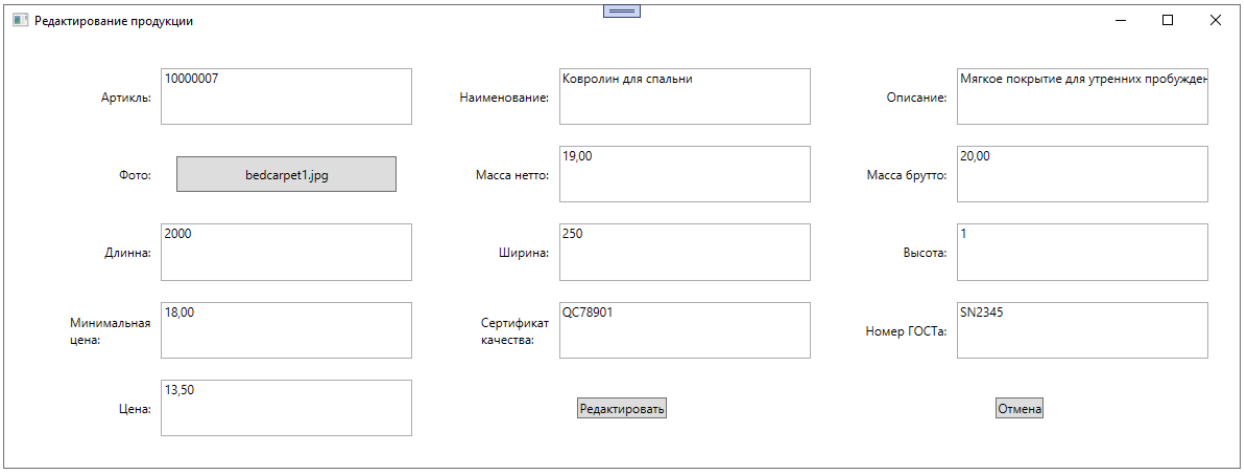
В результате получаем интерфейс окна изменения данных о закупках:

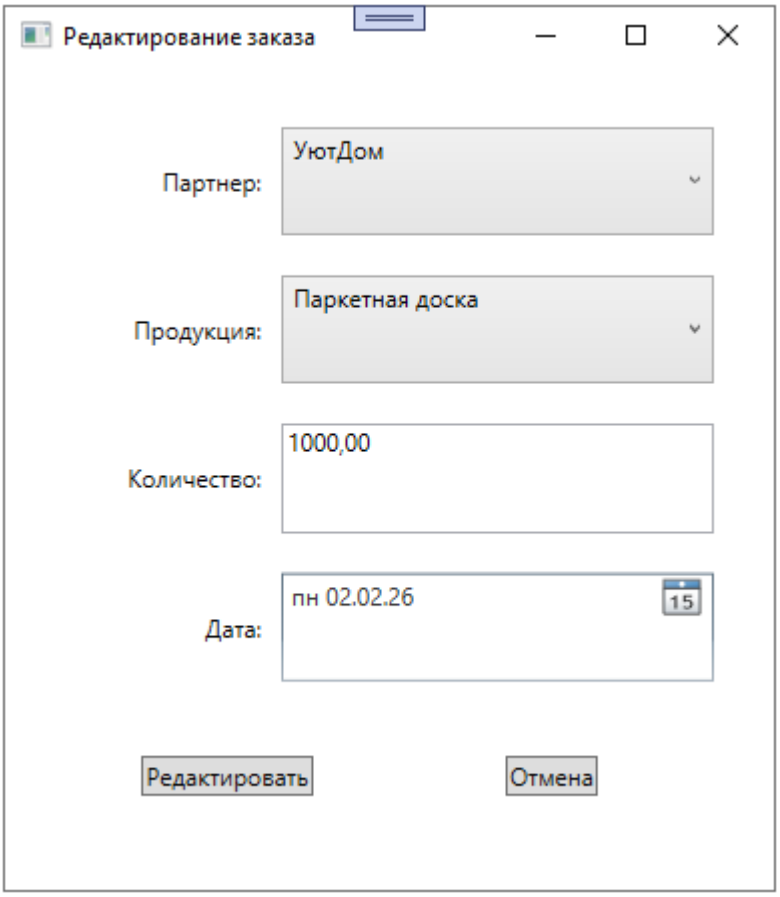


3. В результате получаем интерфейс приложения:




Редактирование партнера
Тип:
ООО
Наименовани
е:
УютДом
Юр. адрес:
625721, г. Тюмень, ул. Парковая, 1
ИНН:
797200381049
Директор:
Петров М. Д.
Телефон:
+79007477589
E-mail:
aquino_uriel90@mail.ru
Логотип:
Рейтинг:
7
Редактировать
Отмена


Редактирование продукции
Артикль:
10000007
Наименование:
Ковролин для спальни
Описание:
Мягкое покрытие для утренних пробужден
Фото:
bedcarpet1.jpg
Масса нетто:
19,00
Масса брутто:
20,00
Длинна:
2000
Ширина:
250
Высота:
1
Минимальная
цена:
18,00
Сертификат
качества:
QC78901
Номер ГОСТа:
SN2345
Цена:
13,50
Редактировать
Отмена


Редактирование заказа
Партнер:
УютДом
Продукция:
Паркетная доска
Количество:
1000,00
Дата:
пн 02.02.26
15
Редактировать
Отмена

Этап 3. Формирование основной логики приложения в .Net Framework

1. Формируем логику действий в MainWindow.xaml.cs.

```
using System.IO;
using System.Text;
using System.Text.Json;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using Application.Services.Services;
using ClassLibrary.Classes;
using ClassLibrary.DTOs;
using MasterFloor.DataForms;
using Microsoft.Win32;
using OfficeOpenXml;

namespace MasterFloor
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        public readonly PartnerService partnerService = new();
        public readonly ProductionService productionService = new();
        public readonly ProductSaleService productSaleService = new();
        public List<Partner> partners = [];
        public List<Production> productions = [];
        public List<ProductSale> productSales = [];
        public List<PartnerDto> partnerDtos = [];
        public List<ProductSalesDto> psti = [];
        public MainWindow()

        private void ReWrite()

        private void ReWrite(Partner partner)

        private void Source_TB_TextChanged(object sender,
System.Windows.Controls.TextChangedEventArgs e)

        private void Partners_LB_Selected(object sender,
System.Windows.Controls.SelectionChangedEventArgs e)

        private void AddPartner(object sender, RoutedEventArgs e)

        private void AddProduction(object sender, RoutedEventArgs e)

        private void AddProductSaleSelected(object sender, RoutedEventArgs e)

        private void AddProductSale(object sender, RoutedEventArgs e)

        private void RedactPartner(object sender, RoutedEventArgs e)

        private void RedactProduction(object sender, RoutedEventArgs e)

        private void RedactProductSale(object sender, RoutedEventArgs e)
```

```
        private void DeletePartner(object sender, RoutedEventArgs e)

        private void DeleteProductSale(object sender, RoutedEventArgs e)

        private void DeleteProduction(object sender, RoutedEventArgs e)

        private void ToExcel(object sender, RoutedEventArgs e)

        private void Save(object sender, RoutedEventArgs e)

        private void ExcelSave(object sender, RoutedEventArgs e)

        private void Load(object sender, RoutedEventArgs e)

        public static RootObject LoadFromExcel(string filePath)

        private void ExcelLoad(object sender, RoutedEventArgs e)
    }
}
```

2. Формируем логику действий в ChoiceWindow.xaml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using ClassLibrary.Classes;

namespace MasterFloor.DataForms
{
    public partial class ChoiceWindow : Window
    {
        public ChoiceWindow(List<Production> productions)
        {
            InitializeComponent();
            foreach (Production p in productions)
            {
                Choice_CB.Items.Add(p.Name);
            }
        }
        public int GetIndex()
        {
            return Choice_CB.SelectedIndex;
        }

        private void accept_btn_Click(object sender, RoutedEventArgs e)
        {
            this.DialogResult = true;
            this.Close();
        }
    }
}
      }
```

3. Формируем логику действий в PartnerRedactWindow.xaml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using ClassLibrary.Classes;
using Microsoft.Win32;

namespace MasterFloor.DataForms
{
    /// <summary>
    /// Логика взаимодействия для PartnerRedactWindow.xaml
    /// </summary>
    public partial class PartnerRedactWindow : Window
    {
        private readonly int Id = 0;
        public PartnerRedactWindow()
        {
            InitializeComponent();
            this.Title = "Добавление партнера";
        }
        public PartnerRedactWindow(Partner partner)
        {
            InitializeComponent();
            this.Title = "Редактирование партнера";
            this.accept_btn.Content = "Редактировать";
            Id = partner.Id;
            Partner_Type_CB.Text = partner.Type;
            Partner_Name_TB.Text = partner.Name;
            Partner_Legal_Address_TB.Text = partner.LegalAddress;
            Partner_INN_TB.Text = partner.Inn;
            Partner_Director_Name_TB.Text = partner.DirectorName;
            Partner_Phone_TB.Text = partner.Phone;
            Partner_Email_TB.Text = partner.Email;
            Partner_Logo_B.Content = partner.Logo;
            Partner_Rating_TB.Text = partner.Rating.ToString();
        }
        public Partner? GetPartner()
        {
            if (!TryGetRating(out int rating))
                return null;

            Partner p = new()
            {
                Type = Partner_Type_CB.Text,
                Name = Partner_Name_TB.Text,
                LegalAddress = Partner_Legal_Address_TB.Text,
                Inn = Partner_INN_TB.Text,
                DirectorName = Partner_Director_Name_TB.Text,
                Phone = Partner_Phone_TB.Text,
                Email = Partner_Email_TB.Text,
                Logo = Partner_Logo_B.Content.ToString(),
```

```
                Rating = rating
            };

            if (Id != 0)
            {
                p.Id = Id;
            }

            return p;
        }

        private bool TryGetRating(out int rating)
        {
            if (int.TryParse(Partner_Rating_TB.Text, out rating))
            {
                return true;
            }
            else
            {
                MessageBox.Show("Неверное значение рейтинга!");
                return false;
            }
        }

        private void accept_btn_Click(object sender, RoutedEventArgs e)
        {
            if (GetPartner() != null)
            {
                this.DialogResult = true;
                this.Close();
            }
        }

        private void Partner_Logo_Choosing(object sender, RoutedEventArgs e)
        {
            var openFileDialog = new OpenFileDialog { Title = "Select a File",
Filter = "Image files (*.png;*.jpeg)|*.png;*.jpeg|All files (*.*)|*.*" };


            if (openFileDialog.ShowDialog() == true)
            {
                Partner_Logo_B.Content = openFileDialog.FileName;
            }
        }

        private void reject_btn_Click(object sender, RoutedEventArgs e)
        {
            this.DialogResult = false;
            this.Close();
        }
    }
}
```

4. Формируем логику действий в ProductionRedactWindow.xaml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
```

```
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using ClassLibrary.Classes;
using Microsoft.Win32;

namespace MasterFloor.DataForms
{
    /// <summary>
    /// Логика взаимодействия для ProductionRedactWindow.xaml
    /// </summary>
    public partial class ProductionRedactWindow : Window
    {
        private readonly int Id = 0;
        public ProductionRedactWindow()
        {
            InitializeComponent();
            this.Title = "Добавление продукции";
        }
        public ProductionRedactWindow(Production production)
        {
            InitializeComponent();
            this.Title = "Редактирование продукции";
            this.accept_btn.Content = "Редактировать";
            Id = production.Id;
            Production_Article_TB.Text = production.Article;
            Production_Name_TB.Text = production.Name;
            Production_Description_TB.Text = production.Description;
            Production_Photo_B.Content = production.Photo;
            Production_Min_Partner_Price_TB.Text =
production.MinPartnerPrice.ToString();
            Production_Length_TB.Text = production.Length.ToString();
            Production_Width_TB.Text = production.Width.ToString();
            Production_Height_TB.Text = production.Height.ToString();
            Production_Net_Weight_TB.Text = production.NetWeight.ToString();
            Production_Gross_Weight_TB.Text = production.GrossWeight.ToString();
            Production_Quality_Certificate_TB.Text =
production.QualityCertificate.ToString();
            Production_Standard_Number_TB.Text =
production.StandardNumber.ToString();
            Production_Cost_Price_TB.Text= production.CostPrice.ToString();
        }

        private bool ValidateInputs()
        {
            if (!double.TryParse(Production_Min_Partner_Price_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение мин. цены!");
                return false;
            }
            if (!int.TryParse(Production_Length_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение длинны!");
                return false;
            }
            if (!int.TryParse(Production_Width_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение ширины!");
                return false;
            }
            if (!int.TryParse(Production_Height_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение высоты!");
```

```
                return false;
            }
            if (!double.TryParse(Production_Net_Weight_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение массы нетто!");
                return false;
            }
            if (!double.TryParse(Production_Gross_Weight_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение массы брутто!");
                return false;
            }

            if (!double.TryParse(Production_Cost_Price_TB.Text, out _))
            {
                MessageBox.Show("Неверное значение цены!");
                return false;
            }

            return true;
        }
        public Production? GetProduction()
        {
            if (ValidateInputs())
            {
                Production p = new()
                {
                    Article = Production_Article_TB.Text,
                    Name = Production_Name_TB.Text,
                    Description = Production_Description_TB.Text,
                    Photo = Production_Photo_B.Content.ToString(),
                    MinPartnerPrice =
Convert.ToDouble(Production_Min_Partner_Price_TB.Text),
                    Length = Convert.ToInt32(Production_Length_TB.Text),
                    Width = Convert.ToInt32(Production_Width_TB.Text),
                    Height = Convert.ToInt32(Production_Height_TB.Text),
                    NetWeight = Convert.ToDouble(Production_Net_Weight_TB.Text),
                    GrossWeight = Convert.ToDouble(Production_Gross_Weight_TB.Text),
                    QualityCertificate = Production_Quality_Certificate_TB.Text,
                    StandardNumber = Production_Standard_Number_TB.Text,
                    CostPrice = Convert.ToDouble(Production_Cost_Price_TB.Text)
                };
                if (Id != 0)
                {
                    p.Id = Id;
                }
                return p;
            }
            return null;
        }

        private void accept_btn_Click(object sender, RoutedEventArgs e)
        {
            if (GetProduction() != null)
            {
                this.DialogResult = true;
                this.Close();
            }
        }

        private void Production_Logo_Choosing(object sender, RoutedEventArgs e)
        {
            var openFileDialog = new OpenFileDialog { Title = "Select a File",
Filter = "Image files (*.png;*.jpeg)|*.png;*.jpeg|All files (*.*)|*.*" };
```

```
            if (openFileDialog.ShowDialog() == true)
            {
                Production_Photo_B.Content = openFileDialog.FileName;
            }
        }

        private void reject_btn_Click(object sender, RoutedEventArgs e)
        {
            this.DialogResult = false;
            this.Close();
        }
    }
}
```

5. Формируем логику действий в ProductSaleRedactWindow.xaml.cs

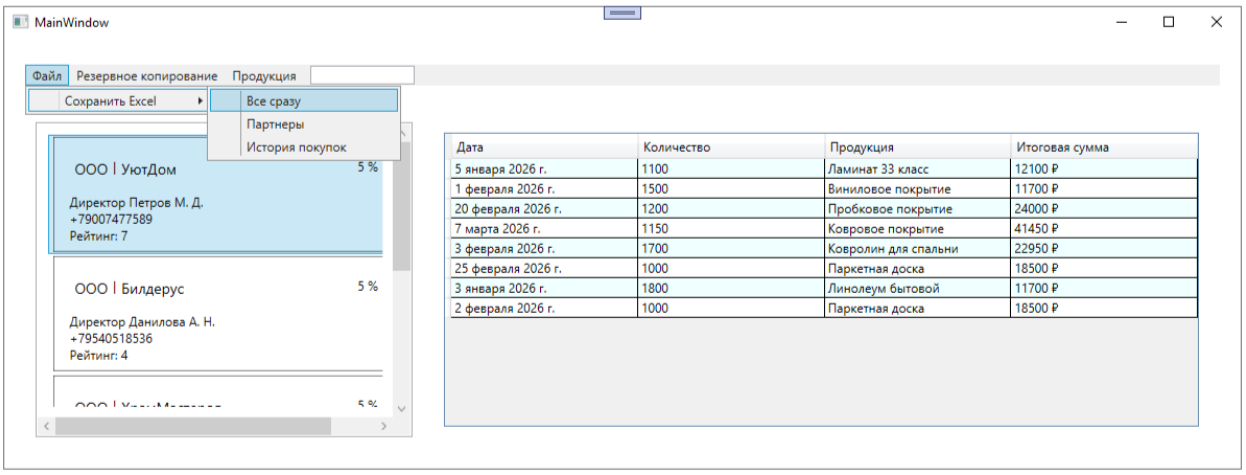
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;
using Application.Services.Services;
using ClassLibrary.Classes;
using ClassLibrary.DTOs;

namespace MasterFloor.DataForms
{
    public partial class ProductSaleRedactWindow : Window
    {
        private readonly List<Partner> partners;
        private readonly List<Production> productions;
        private readonly List<ProductSale> productSales;
        private readonly int Id = 0;
        public ProductSaleRedactWindow(List<Partner> partners, List<Production>
productions, List<ProductSale> productSales)
        {
            InitializeComponent();
            this.Title = "Добавление заказа";
            this.partners = partners;
            this.productions = productions;
            this.productSales = productSales;
            foreach (Partner p in partners)
            {
                ProductSale_Partner_CB.Items.Add(p.Name);
            }
            foreach (Production p in productions)
            {
                ProductSale_Production_CB.Items.Add(p.Name);
            }
        }
        public ProductSaleRedactWindow(int partnerindex, List<Partner> partners,
List<Production> productions, List<ProductSale> productSales)
        {
            InitializeComponent();
```

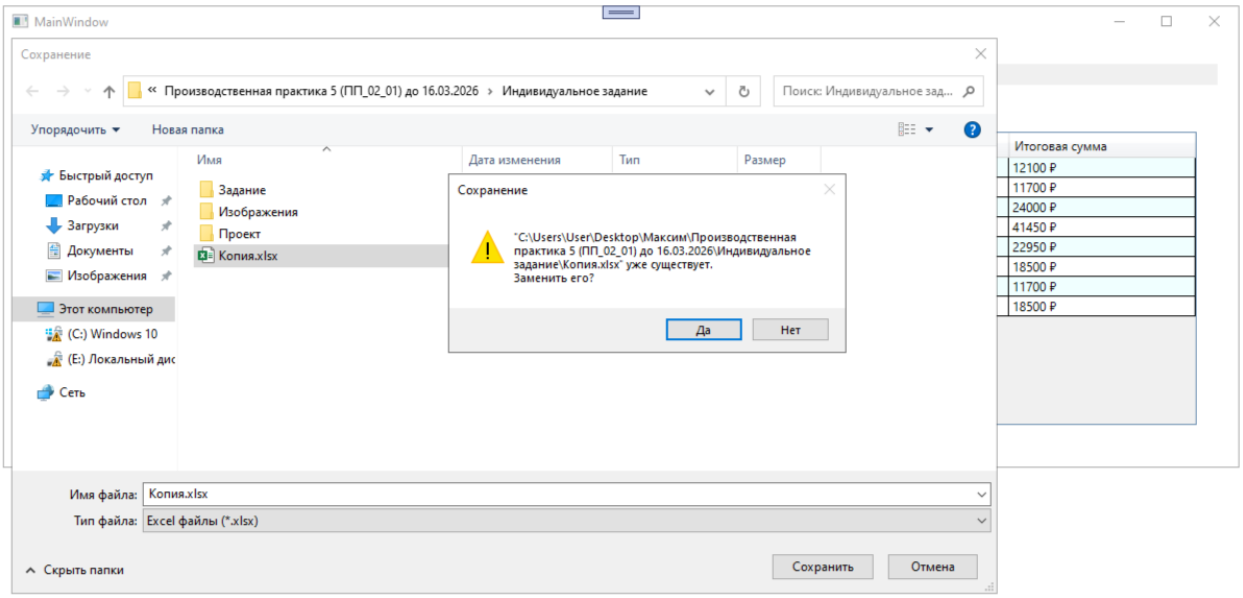
```
            this.Title = "Добавление заказа";
            this.partners = partners;
            this.productions = productions;
            this.productSales = productSales;
            foreach (Partner p in partners)
            {
                ProductSale_Partner_CB.Items.Add(p.Name);
            }
            foreach (Production p in productions)
            {
                ProductSale_Production_CB.Items.Add(p.Name);
            }
            ProductSale_Partner_CB.SelectedIndex = partnerindex;
        }
        public ProductSaleRedactWindow(ProductSale productSale, List<Partner>
partners, List<Production> productions, List<ProductSale> productSales)
        {
            InitializeComponent();
            this.Title = "Редактирование заказа";
            this.accept_btn.Content = "Редактировать";
            this.partners = partners;
            this.productions = productions;
            this.productSales = productSales;
            foreach (Partner p in partners)
            {
                ProductSale_Partner_CB.Items.Add(p.Name);
            }
            foreach (Production p in productions)
            {
                ProductSale_Production_CB.Items.Add(p.Name);
            }
            Id = productSale.Id;
            ProductSale_Count_TB.Text = productSale.Count.ToString();
            ProductSale_Date_DP.SelectedDate =
productSale.SaleDate.ToDateTime(TimeOnly.Parse("10:00 PM"));
            ProductSale_Partner_CB.SelectedIndex = partners.FindIndex(x => x.Id ==
productSale.PartnerId);
            ProductSale_Production_CB.SelectedIndex = productions.FindIndex(x =>
x.Id == productSale.ProductionId);
        }
        private bool ValidateInputs()
        {
            Partner partner = partners[ProductSale_Partner_CB.SelectedIndex];
            Production production =
productions[ProductSale_Production_CB.SelectedIndex];

            if (partner == null)
            {
                MessageBox.Show("Данный учитель(id: " + partner?.Id + ") не
найден!");
                return false;
            }

            if (production == null)
            {
                MessageBox.Show("Данная работа(id: " + production?.Id + ") не
найдена!");
                return false;
            }

            if (!int.TryParse(ProductSale_Count_TB.Text, out _))
            {
                MessageBox.Show("Неправильное количество продукции!");
                return false;
            }
```

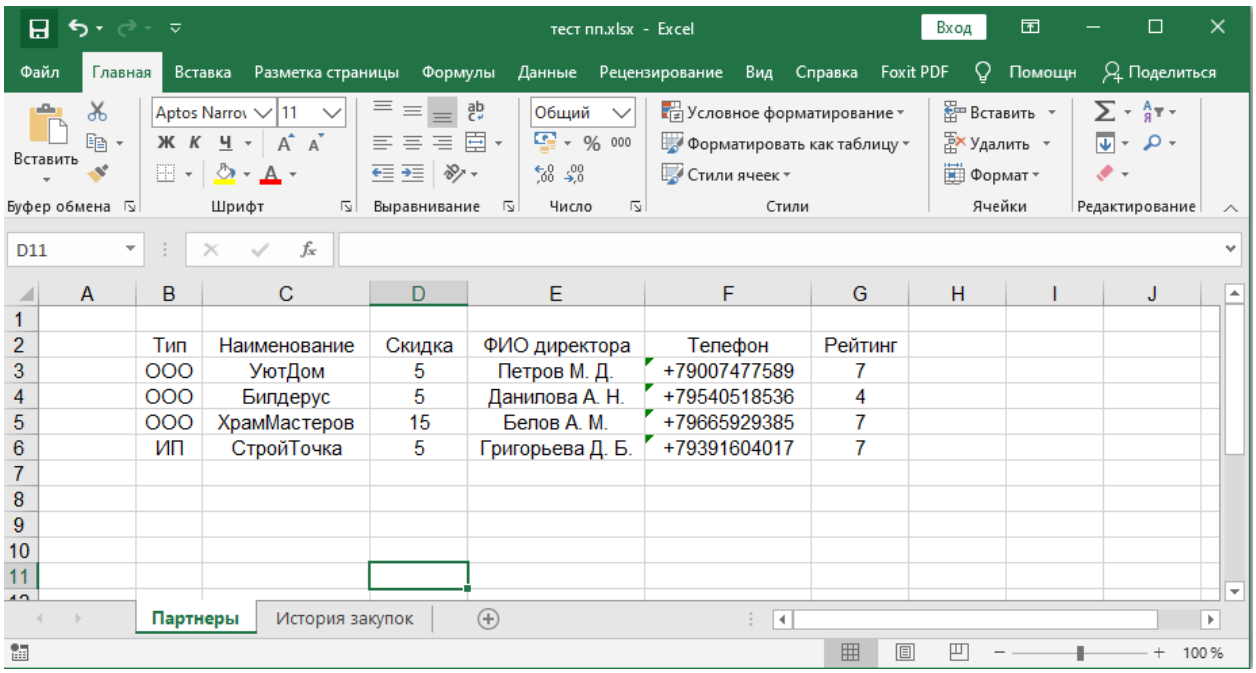
```
            if (ProductSale_Date_DP.SelectedDate == null)
            {
                MessageBox.Show("Неправильная дата!");
                return false;
            }

            return true;
        }

        public ProductSale? GetProductSale()
        {
            if (!ValidateInputs())
                return null;

            Partner partner = partners[ProductSale_Partner_CB.SelectedIndex];
            Production production =
productions[ProductSale_Production_CB.SelectedIndex];

            int count = int.Parse(ProductSale_Count_TB.Text);

            double TotalPrice;
            if (production.CostPrice * partner.GetDiscount(productSales) <
production.MinPartnerPrice)
                TotalPrice = count * production.MinPartnerPrice;
            else
                TotalPrice = count * production.CostPrice.GetValueOrDefault() *
partner.GetDiscount(productSales);

            #pragma warning disable CS8629 // Тип значения, допускающего NULL, может
быть NULL.
            DateTime selectedDate = (DateTime)ProductSale_Date_DP.SelectedDate;
            #pragma warning restore CS8629 // Тип значения, допускающего NULL, может
быть NULL.

            ProductSale p = new()
            {
                PartnerId = partner.Id,
                ProductionId = production.Id,
                Count = count,
                SaleDate = DateOnly.FromDateTime(selectedDate),
                TotalPrice = TotalPrice
            };
            if (Id != 0)
                p.Id = Id;
            return p;
        }

        private void accept_btn_Click(object sender, RoutedEventArgs e)
        {
            if (GetProductSale() != null)
            {
                this.DialogResult = true;
                this.Close();
            }
        }

        private void reject_btn_Click(object sender, RoutedEventArgs e)
        {
            this.DialogResult = false;
            this.Close();
        }
    }
  }
```

6. Проверяем работоспособность написанной программы:

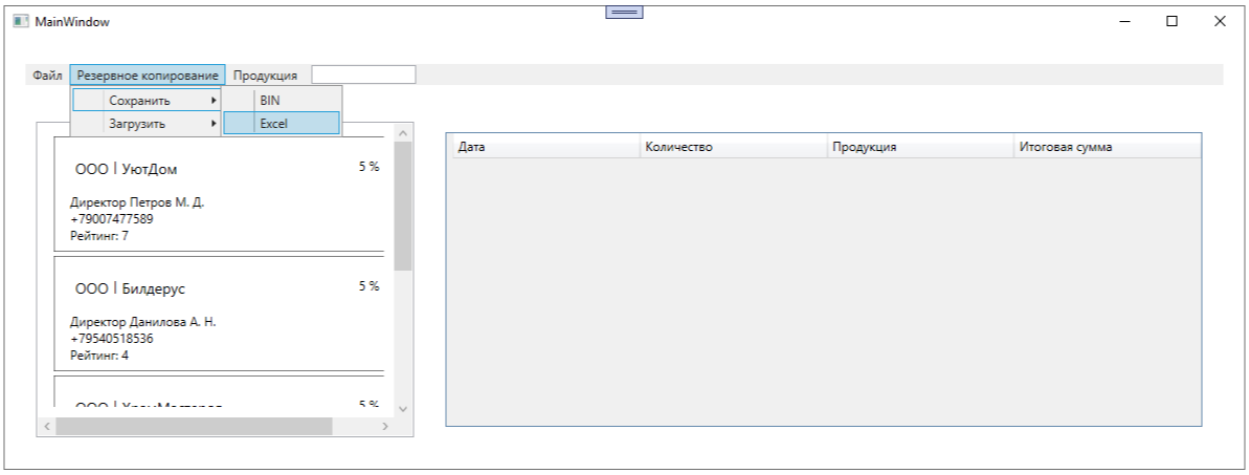
6.1. Проверяем выгрузку данных в Excel. При последовательном нажатии нопок «Файл» - «Сохранить Excel» «Все сразу/Партнеры/История покупок» получаем отчет в виде Excel-файла (обеих таблиц при нажатии кнопки «Все сразу», только Партнеров при нажатии кнопки «Партнеры», и только Историю закупок выбранного партнера при нажатии кнопки «История покупок»).

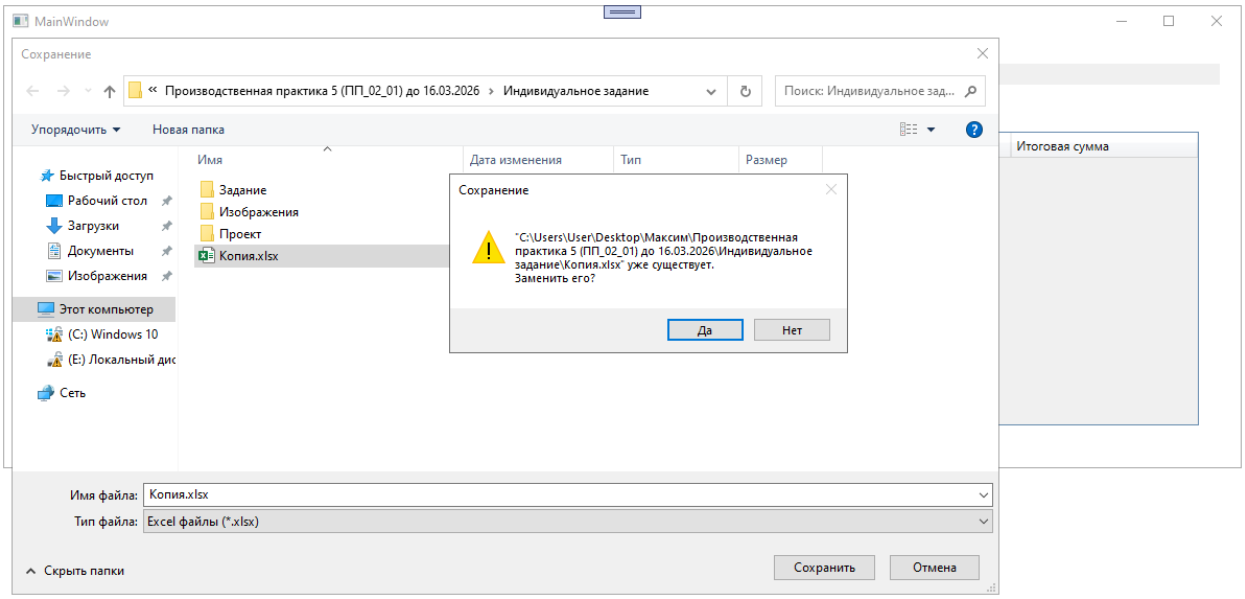


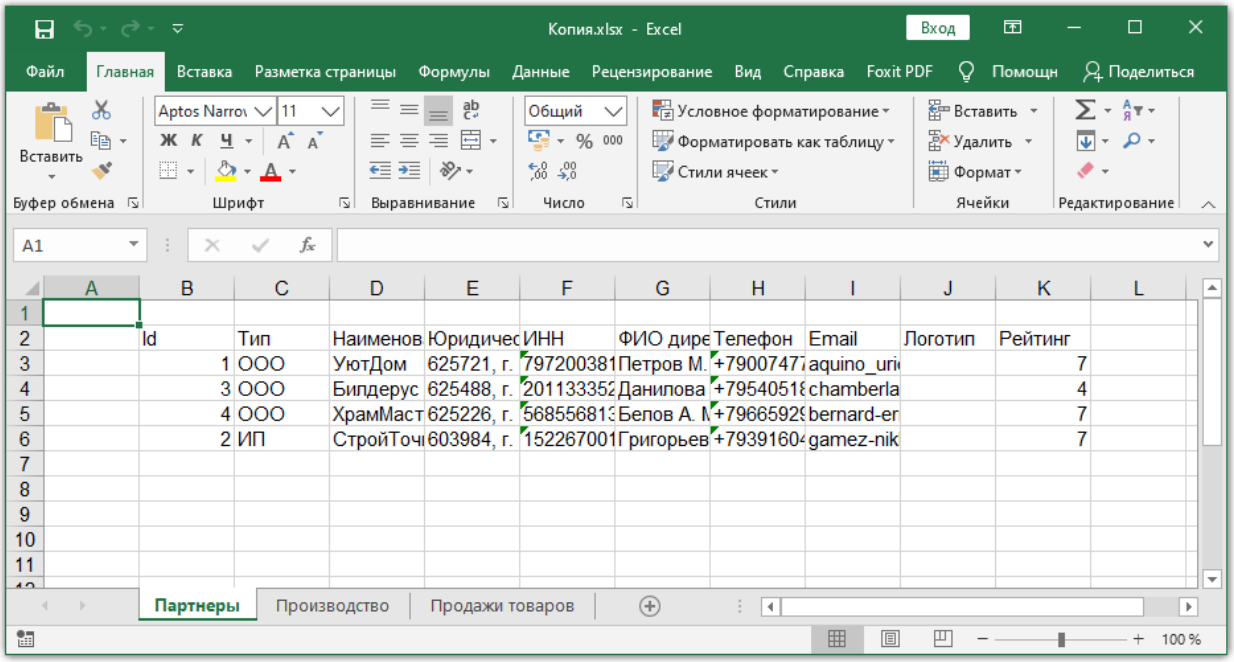




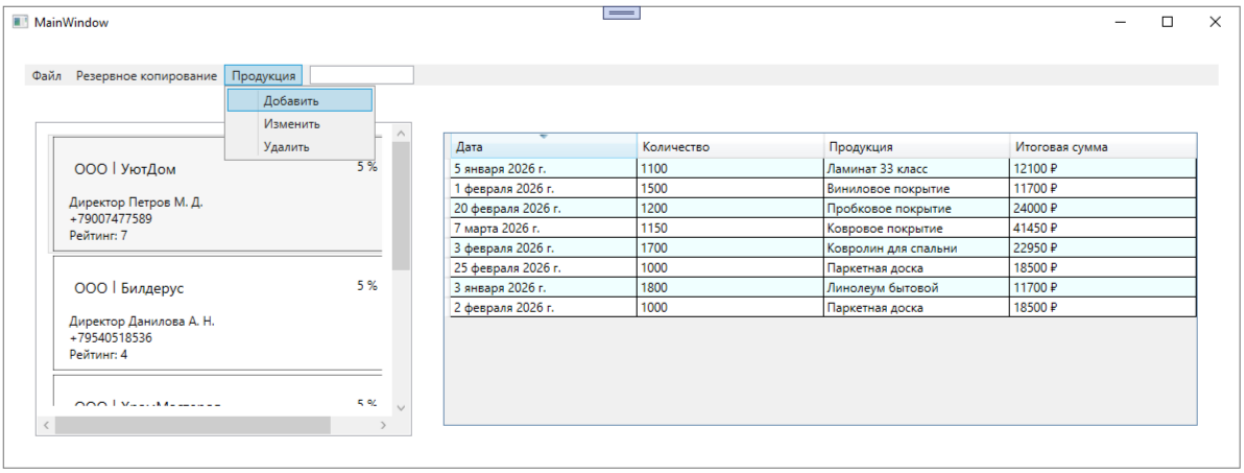
6.2. Проверяем загурзку данных в приложение. При нажатии кнопки «Резервное копирование» получаем (при дальнейшем нажатии кнопки «Сохранить») или загружаем (при дальнейшем нажатии кнопки «Загрузить») резервную копию в виде файла BIN (при дальнейшем нажатии кнопки «BIN») или Excel (при дальнейшем нажатии кнопки «Excel»).

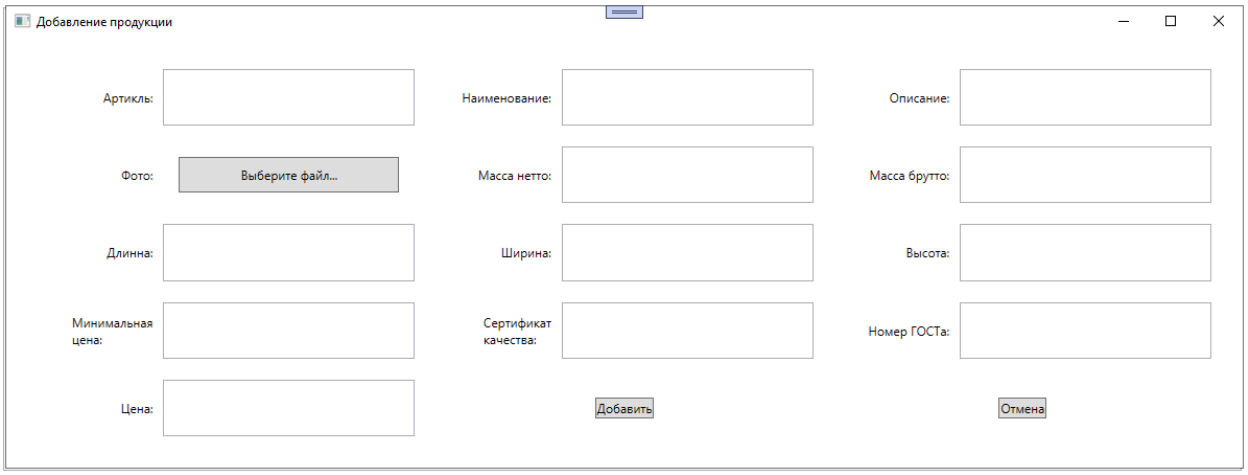


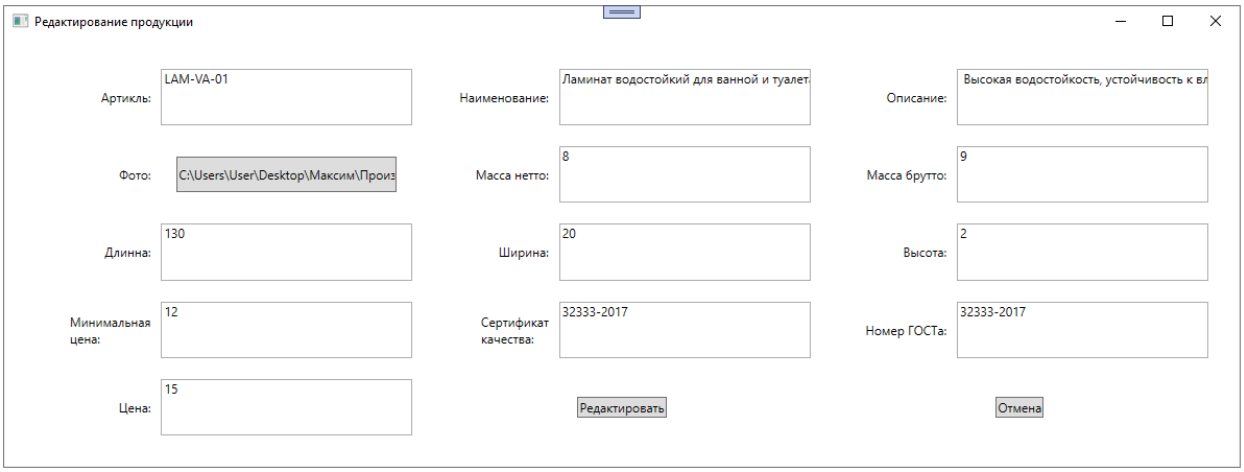


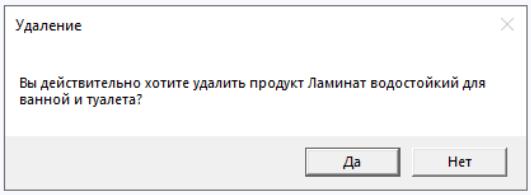


6.3. Проверяем функции по работе с данными о продукции. При нажатии кнопки «Продукция» можно добавить (при нажатии кнопки «Добавить»), а также изменить (при нажатии кнопки «Изменить») и удалить (при нажатии кнопки «Удалить») продукцию/данные о продукции.

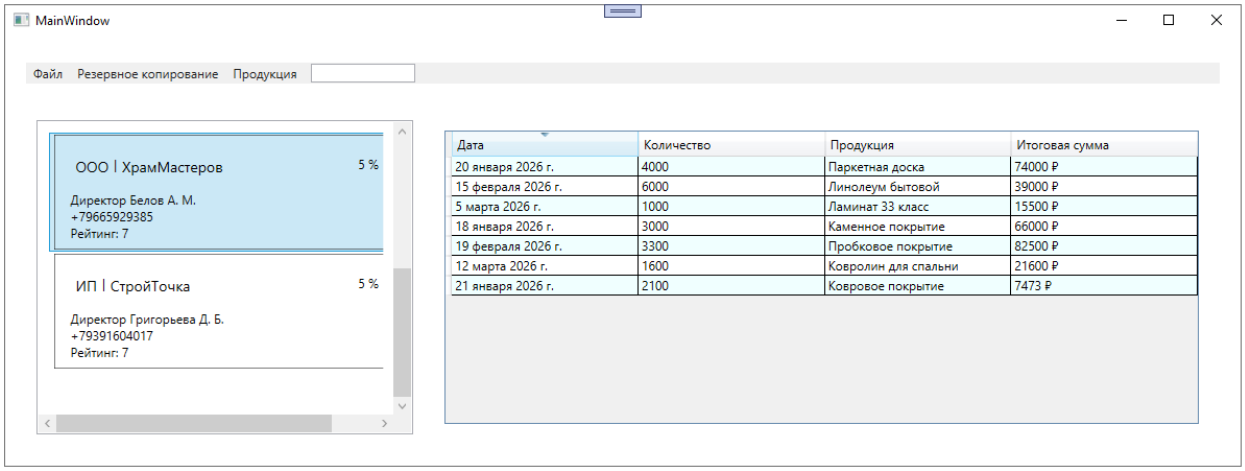


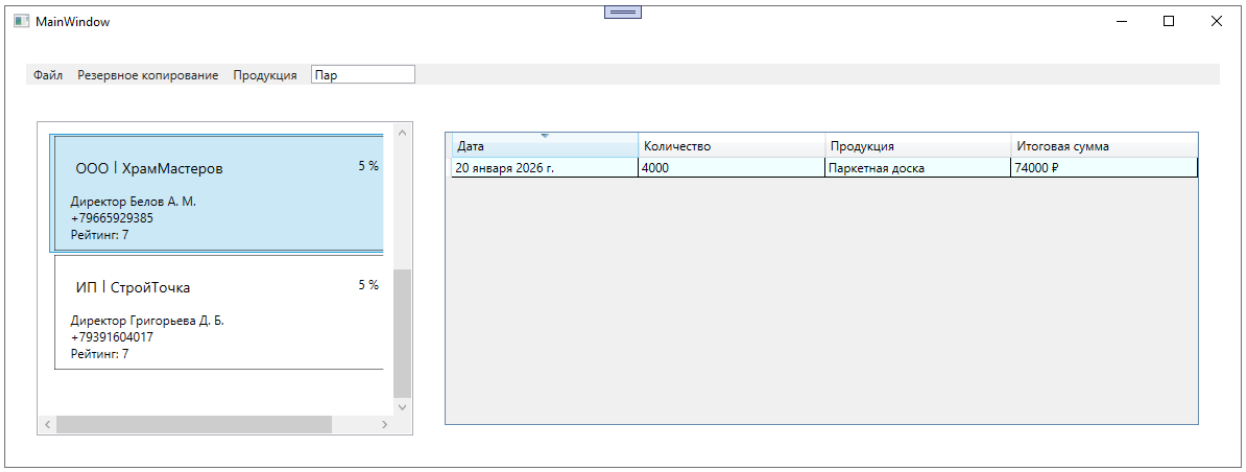




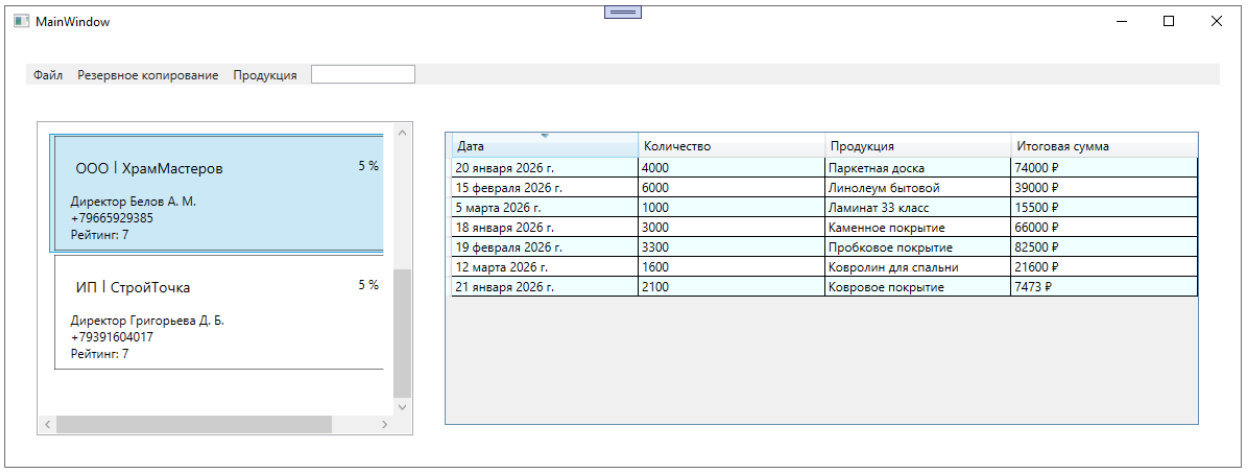


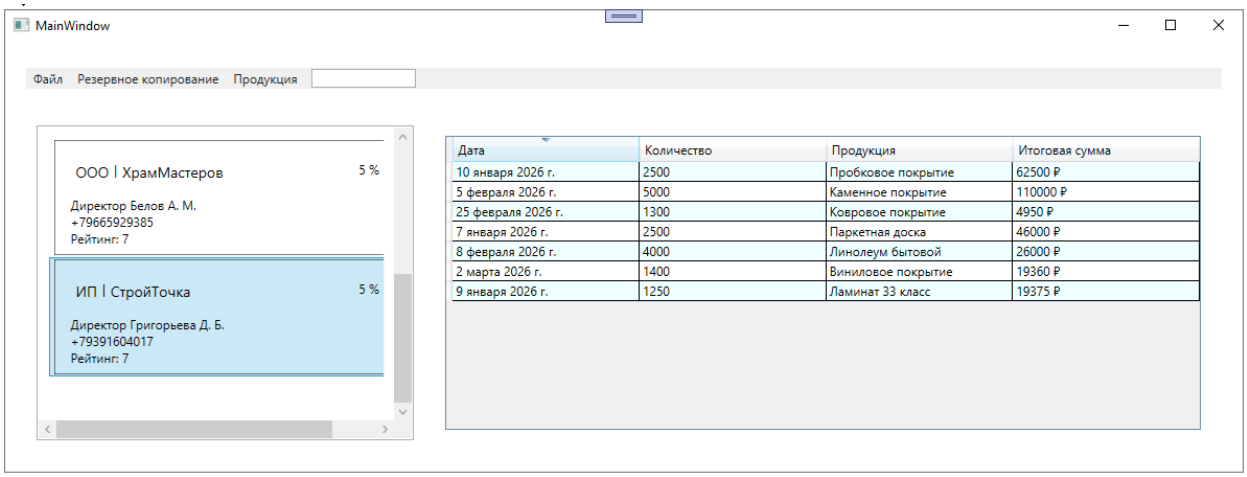
6.4. Провека функции поиска. При введении значения в окно поиска, в таблице с историей закупок отобразятся искомые данные.



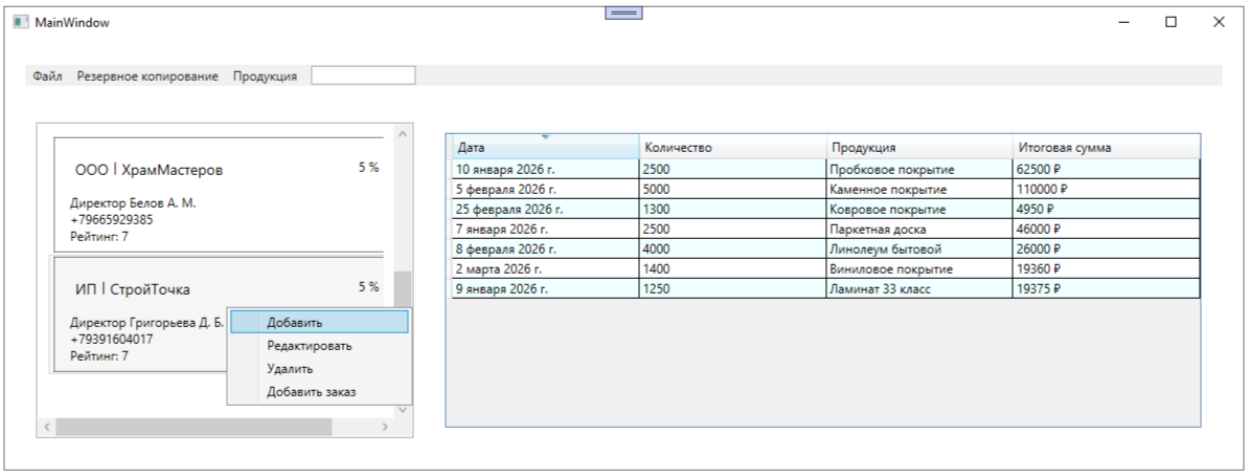


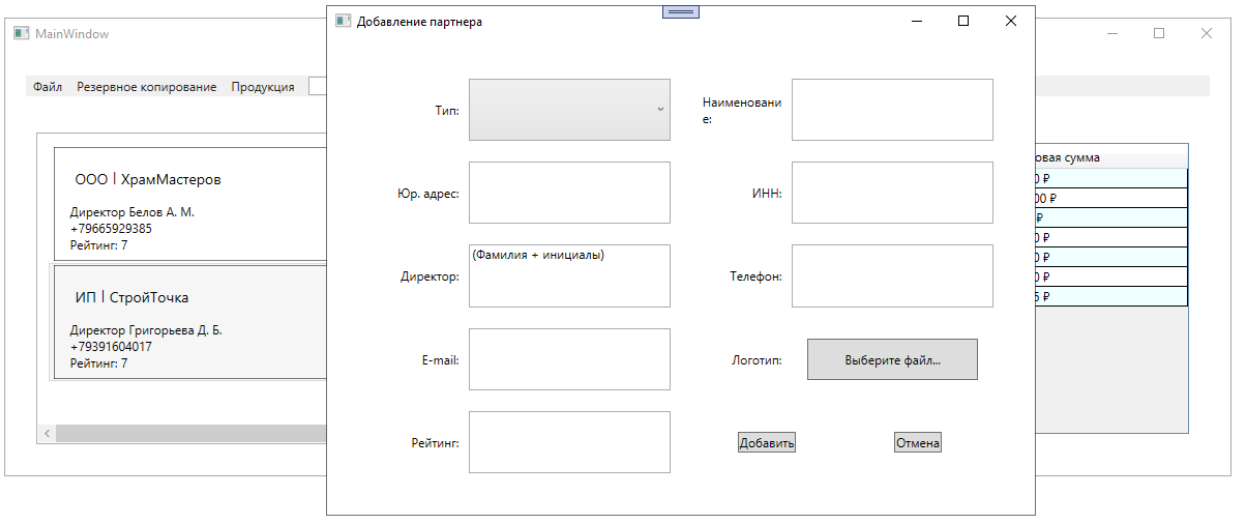
6.5. Проверка функции просмотра истории реализации продукции партнеру. При нажатии на партнера в списке партнеров в таблицу будет выведен список заказов выбранного партнера.

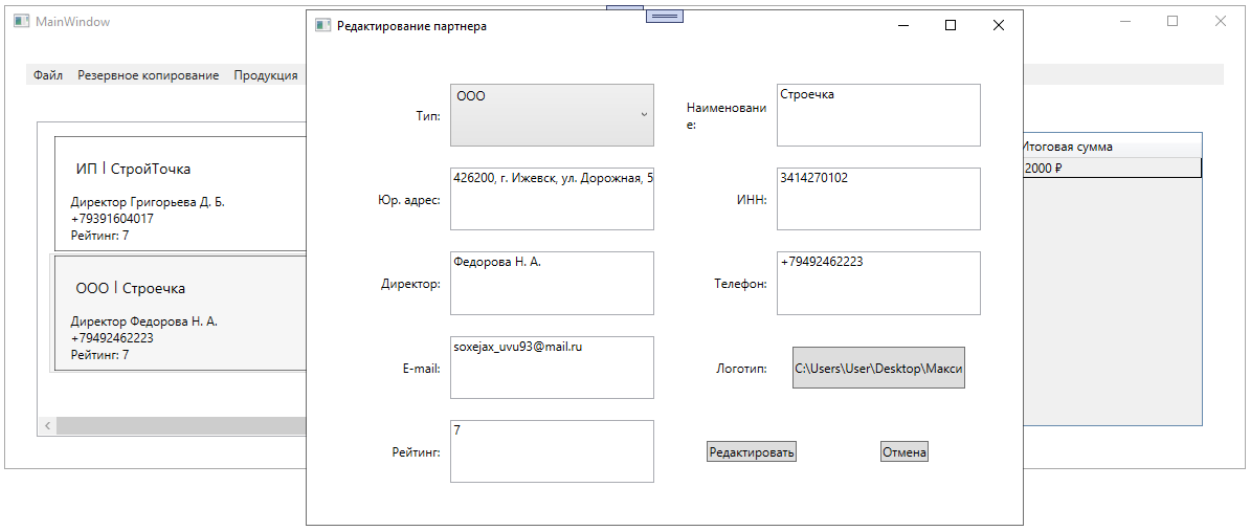


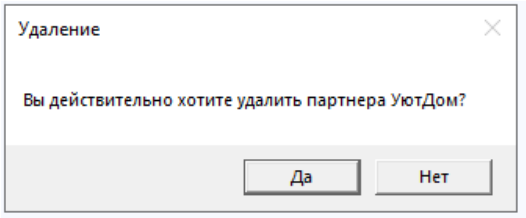


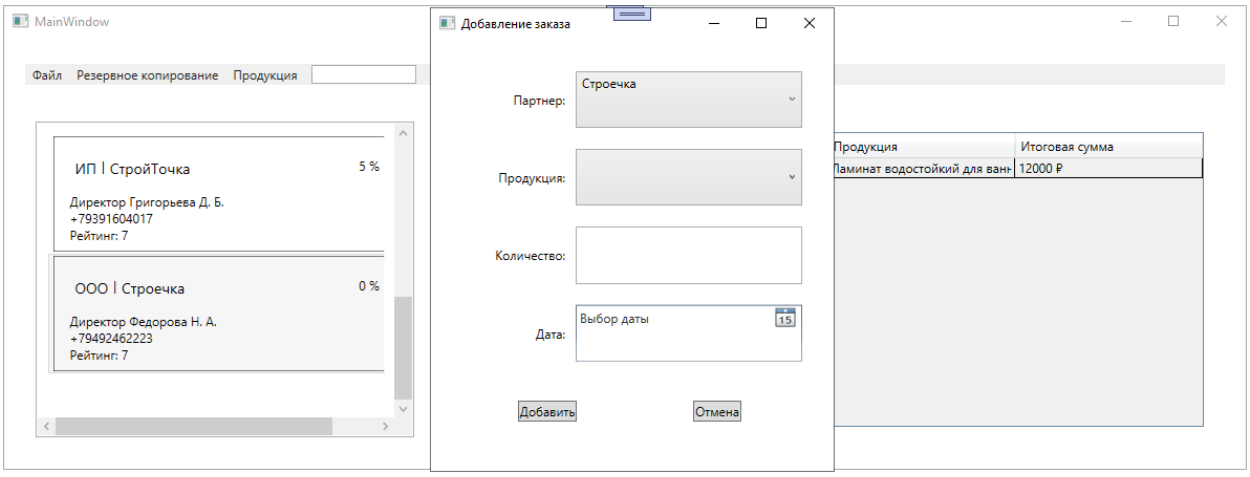
6.6. Проверяем функции по работе с данными о партнере. В контекстном меню списка партнеров при нажатии правой кнопкой мыши можно добавить (при выборе «Добавить»), изменить (при выборе «Редактировать») и удалить (при выборе «Удалить») партнера/данных о партнере, а также добавить запись (при выборе «Добавить заказ») в историю закупок выбранного партнера.



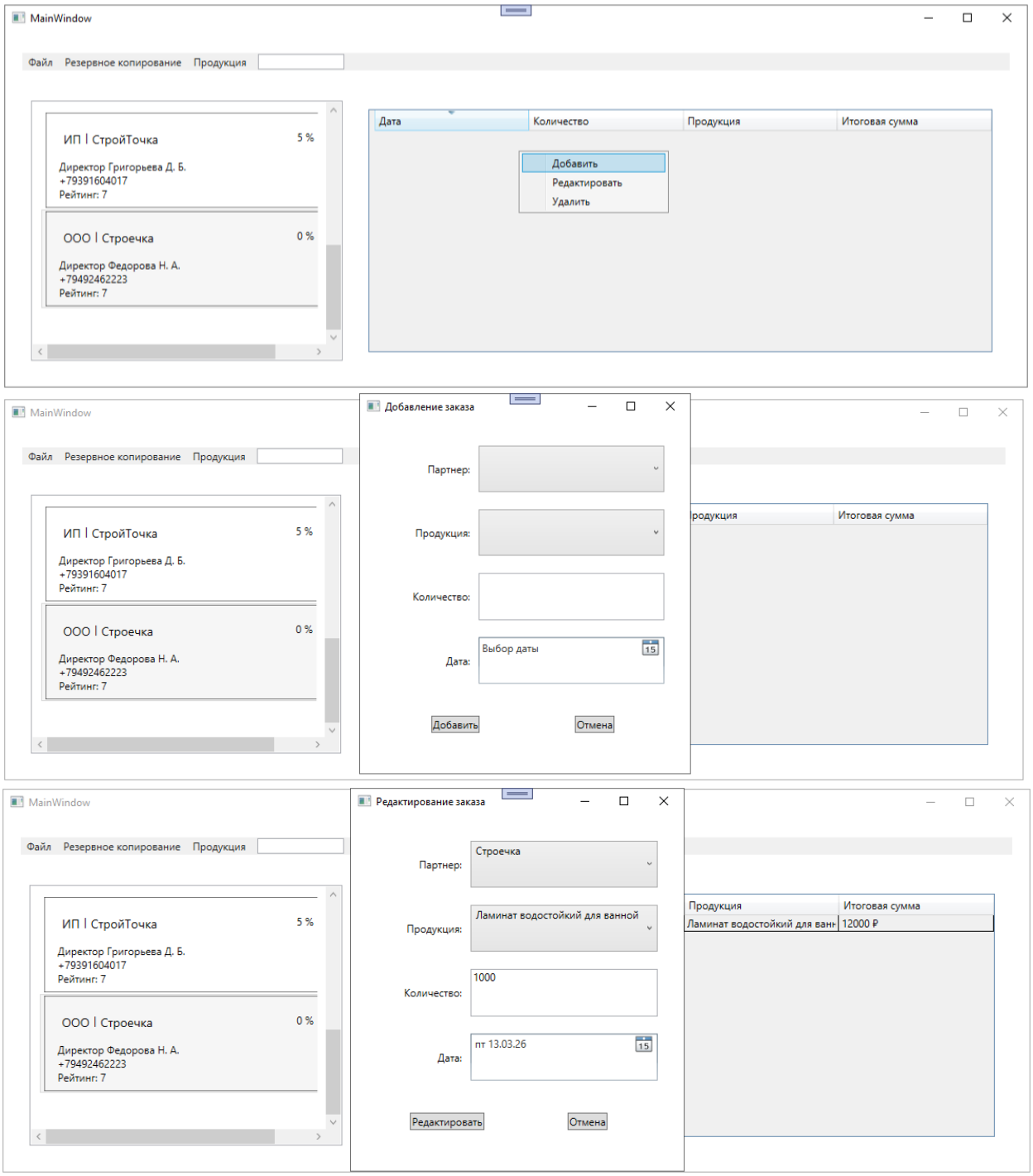

MainWindow
Файл
Резервное копирование
Продукция
ООО | ХрамМастеров
Директор Белов А. М.
+79665929385
Рейтинг: 7
ИП | СтройТочка
Директор Григорьева Д. Б.
+79391604017
Рейтинг: 7
Добавление партнера
Тип:
Наименование:
Юр. адрес:
ИНН:
Директор:
(Фамилия + инициалы)
Телефон:
E-mail:
Логотип:
Выберите файл...
Рейтинг:
Добавить
Отмена

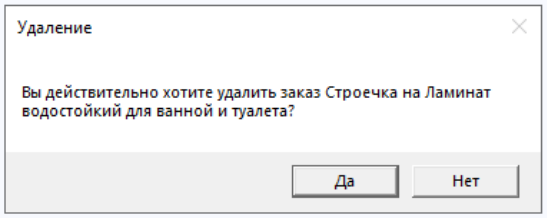

MainWindow
Файл
Резервное копирование
Продукция
ИП | СтройТочка
Директор Григорьева Д. Б.
+79391604017
Рейтинг: 7
ООО | Строечка
Директор Федорова Н. А.
+79492462223
Рейтинг: 7
Редактирование партнера
Тип:
ООО
Наименование:
Строечка
Юр. адрес:
426200, г. Ижевск, ул. Дорожная, 5
ИНН:
3414270102
Директор:
Федорова Н. А.
Телефон:
+79492462223
E-mail:
soxejax_uvu93@mail.ru
Логотип:
C:\Users\User\Desktop\Макси
Рейтинг:
7
Редактировать
Отмена
Итоговая сумма
2000 ₽


Удаление
Вы действительно хотите удалить партнера УютДом?
Да
Нет

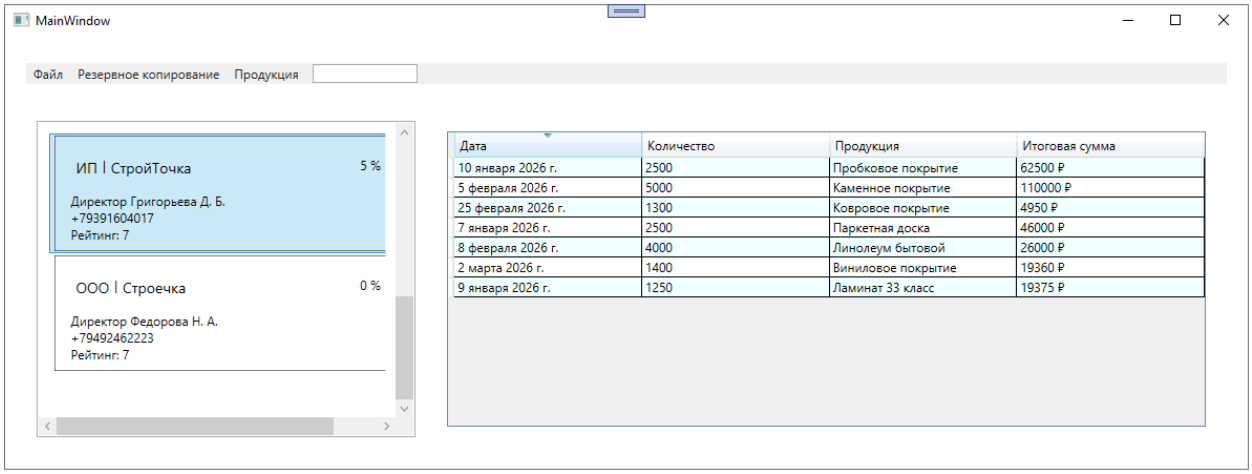

MainWindow
Файл
Резервное копирование
Продукция
ИП | СтройТочка
5 %
Директор Григорьева Д. Б.
+79391604017
Рейтинг: 7
ООО | Строечка
0 %
Директор Федорова Н. А.
+79492462223
Рейтинг: 7
Добавление заказа
Партнер:
Строечка
Продукция:
Количество:
Дата:
Выбор даты
Добавить
Отмена
Продукция
Итоговая сумма
Ламинат водостойкий для ванн
12000 ₽

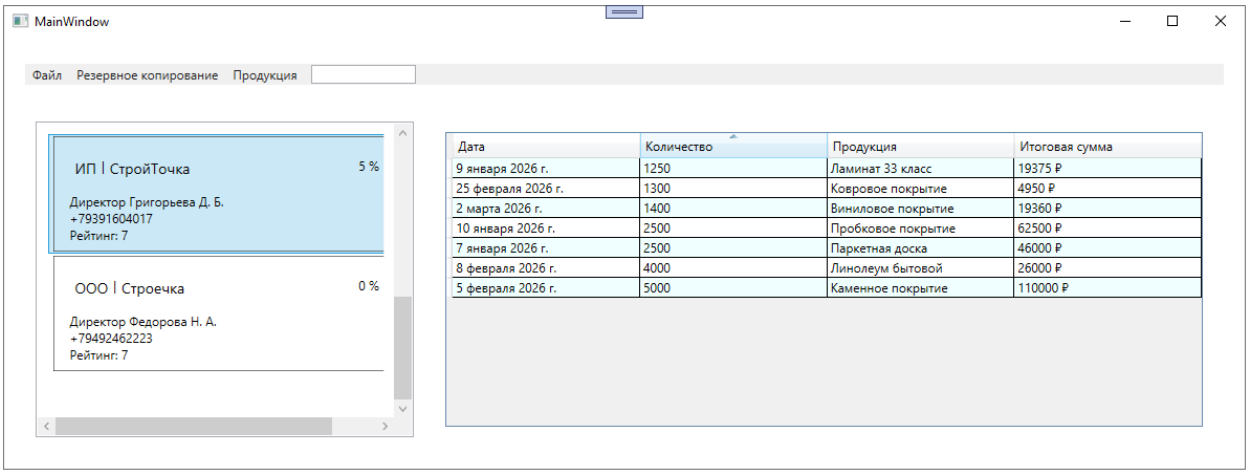
6.7. Проверяем функции по работе с данными о закупках. В контекстном меню таблицы истории закупок партнеров можно добавить (при выборе «Добавить»), изменить (при выборе «Редактировать») и удалить (при выборе «Удалить») закупку/данные о закупке. При изменении и удалении закупки необходимо выбрать нужную закупку.





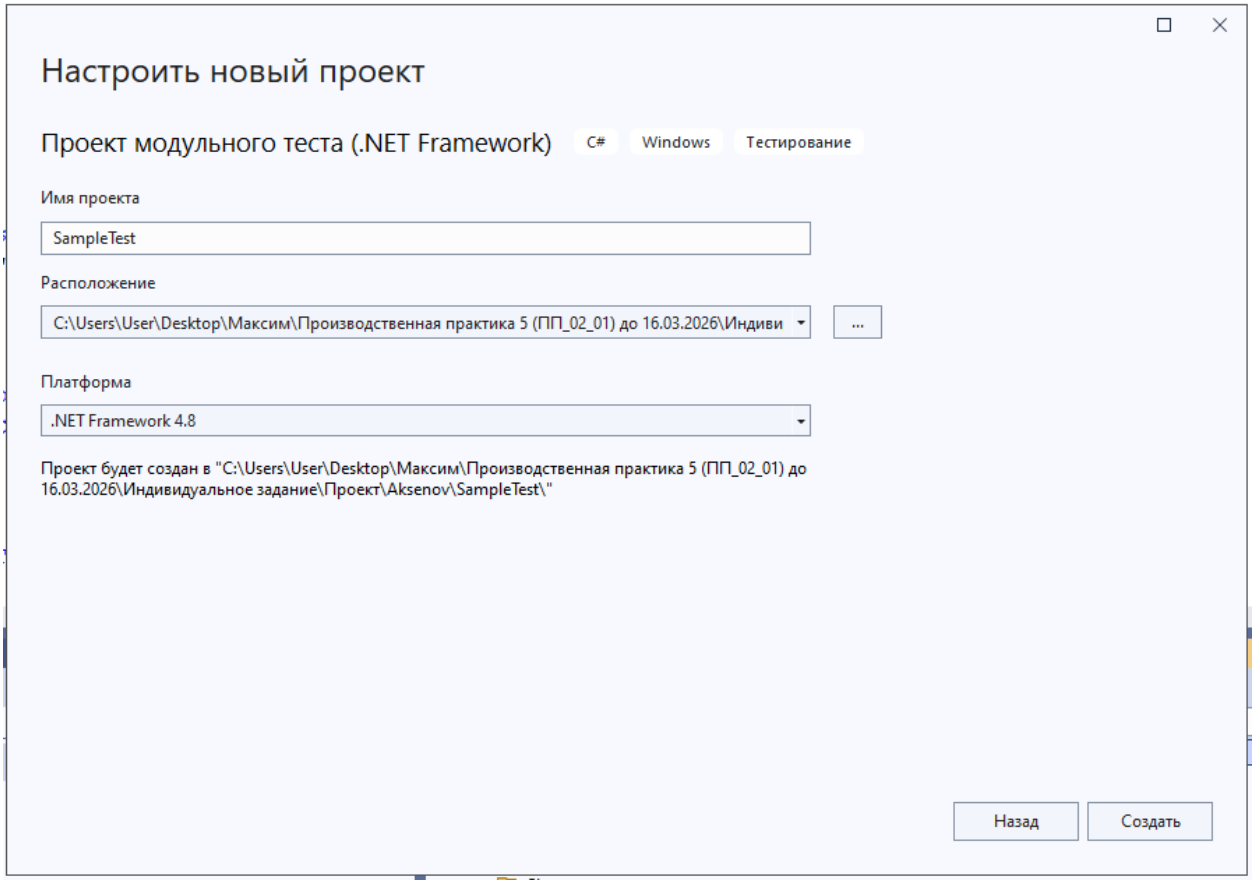
6.8. Проверка функции сортировка. При нажтии на заголовки таблицы истории закупок данные в таблице будут отсортированы.





Этап 4. Создание модульных тестов.

1. Создаем проект модульных тестов.



2. Приступаем к написанию модульных тестов.

```
using OfficeOpenXml;
using ClassLibrary.Classes;
using ClassLibrary.DTOs;
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace TestSample
{
    [TestClass]
    public class MarketExcelGeneratorTests
    {
        private static TableReport CreateTestTableReport()
        {
            return new TableReport
            {
                partnerDtos = new List<PartnerDto>
                {
                    new PartnerDto { Type = "Type1", Name = "Partner1", Discount =
10, DirectorName = "Director1", Phone = "123", Rating = 4 },
                    new PartnerDto { Type = "Type2", Name = "Partner2", Discount =
15, DirectorName = "Director2", Phone = "456", Rating = 4 },
                },
                salesDtos = new List<ProductSalesDto>
                {
                    new ProductSalesDto { Date = "2024-01-01", Count = 5, Production
= "ProdA", TotalPrice = 1000 },
```

```
                    new ProductSalesDto { Date = "2024-01-02", Count = 3, Production
= "ProdB", TotalPrice = 600 },
                }
            };
        }

        [TestMethod]
        public void Generate_ShouldCreatePartnerAndSalesSheets_WhenIsInOneFileIs0()
        {
            // Arrange
            var report = CreateTestTableReport();

            // Act
            var bytes = MarketExcelGenerator.Generate(report, 0);

            // Assert
            using (var stream = new MemoryStream(bytes))
            using (var package = new ExcelPackage(stream))
            {
                // Проверка листа "Партнеры"
                var sheet = package.Workbook.Worksheets["Партнеры"];
                Assert.IsNotNull(sheet);
                Assert.AreEqual("Тип", sheet.Cells["B2"].Text);
                Assert.AreEqual("Partner1", sheet.Cells["C3"].Text);
                Assert.AreEqual(10, sheet.Cells["D3"].GetValue<int>());
                Assert.AreEqual("Director1", sheet.Cells["E3"].Text);

                // Проверка листа "История закупок"
                var salesSheet = package.Workbook.Worksheets["История закупок"];
                Assert.IsNotNull(salesSheet);
                Assert.AreEqual("Дата", salesSheet.Cells["B2"].Text);
                Assert.AreEqual("2024-01-01", salesSheet.Cells["B3"].Text);
                Assert.AreEqual(5, salesSheet.Cells["C3"].GetValue<int>());
            }
        }

        [TestMethod]
        public void Generate_ShouldCreateOnlyPartnerSheet_WhenIsInOneFileIs1()
        {
            var report = CreateTestTableReport();

            var bytes = MarketExcelGenerator.Generate(report, 1);

            using (var stream = new MemoryStream(bytes))
            using (var package = new ExcelPackage(stream))
            {
                Assert.IsNotNull(package.Workbook.Worksheets["Партнеры"]);
                Assert.IsNull(package.Workbook.Worksheets["История закупок"]);
            }
        }

        [TestMethod]
        public void Generate_ShouldCreateBothSheets_WhenIsInOneFileIs2()
        {
            var report = CreateTestTableReport();

            var bytes = MarketExcelGenerator.Generate(report, 2);

            using (var stream = new MemoryStream(bytes))
            using (var package = new ExcelPackage(stream))
            {
                Assert.IsNotNull(package.Workbook.Worksheets["Партнеры"]);
                Assert.IsNotNull(package.Workbook.Worksheets["История закупок"]);
            }
        }
```

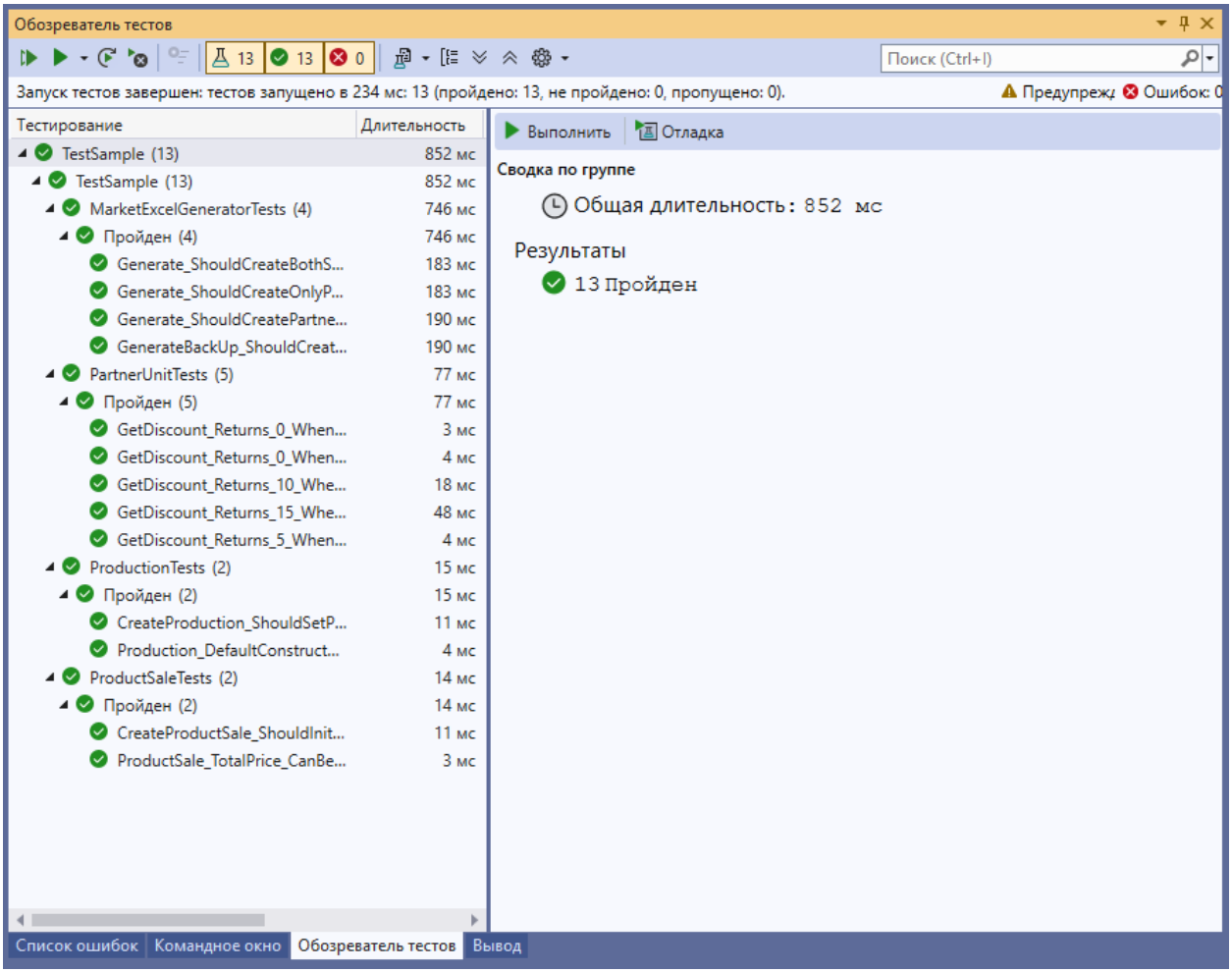
```
        [TestMethod]
        public void GenerateBackUp_ShouldCreateAllBackupSheets()
        {
            // Arrange
            var rootObject = new RootObject
            {
                Partners = new List<Partner>
                {
                    new Partner { Id=1, Type="TypeA", Name="PartnerA",
LegalAddress="Addr1", Inn="123456", DirectorName="DirA", Phone="987",
Email="a@b.com", Logo="logo.png", Rating=4 }
                },
                Productions = new List<Production>
                {
                    new Production { Id=1, Article="A001", Name="ProdName",
Description="desc", Photo="photo.png", MinPartnerPrice=100, Length=10, Width=5,
Height=2, NetWeight=1.5, GrossWeight=2.0, QualityCertificate="Cert",
StandardNumber="Std", CostPrice=50 }
                },
                Products = new List<ProductSale>
                {
                    new ProductSale { Id=1, PartnerId=1, ProductionId=1, Count=10,
SaleDate= new DateOnly(2024, 1, 15), TotalPrice=1000 }
                }
            };

            // Act
            var bytes = MarketExcelGenerator.GenerateBackUp(rootObject);

            // Assert
            using (var stream = new MemoryStream(bytes))
            using (var package = new ExcelPackage(stream))
            {
                Assert.IsNotNull(package.Workbook.Worksheets["Партнеры"]);
                Assert.IsNotNull(package.Workbook.Worksheets["Производство"]);
                Assert.IsNotNull(package.Workbook.Worksheets["Продажи товаров"]);

                var partnerSheet = package.Workbook.Worksheets["Партнеры"];
                Assert.AreEqual("Id", partnerSheet.Cells["B2"].Text);
                Assert.AreEqual(1, partnerSheet.Cells["B3"].GetValue<int>());

                var productionSheet = package.Workbook.Worksheets["Производство"];
                Assert.AreEqual(1, productionSheet.Cells["B3"].GetValue<int>());

                var salesSheet = package.Workbook.Worksheets["Продажи товаров"];
                Assert.AreEqual(1, salesSheet.Cells["B3"].GetValue<int>());
            }
        }
    }

using Microsoft.VisualStudio.TestTools.UnitTesting;
using System.Collections.Generic;
using ClassLibrary.Classes;

namespace TestSample
{
    [TestClass]
    public class PartnerUnitTests
    {
        // Помогает создать экземпляр Partner с заданным Id
        private static Partner CreatePartner(int id)
        {
            return new Partner { Id = id };
        }
```

```
        // Создает список ProductSale для тестирования
        private static List<ProductSale> CreateSales(int partnerId, int count)
        {
            var sales = new List<ProductSale>();
            for (int i = 0; i < count; i++)
            {
                sales.Add(new ProductSale { PartnerId = partnerId, Count = 1 });
            }
            return sales;
        }

        [TestMethod]
        public void GetDiscount_Returns_0_When_TotalProducts_Less_Than_10000()
        {
            var partner = CreatePartner(1);
            var sales = CreateSales(1, 9999);
            var result = partner.GetDiscount(sales);
            Assert.AreEqual(0, result);
        }

        [TestMethod]
        public void
GetDiscount_Returns_5_When_TotalProducts_Between_10000_And_50000()
        {
            var partner = CreatePartner(2);
            var sales = CreateSales(2, 15000);
            var result = partner.GetDiscount(sales);
            Assert.AreEqual(5, result);
        }

        [TestMethod]
        public void
GetDiscount_Returns_10_When_TotalProducts_Between_50000_And_300000()
        {
            var partner = CreatePartner(3);
            var sales = CreateSales(3, 100000);
            var result = partner.GetDiscount(sales);
            Assert.AreEqual(10, result);
        }

        [TestMethod]
        public void GetDiscount_Returns_15_When_TotalProducts_Greater_Than_300000()
        {
            var partner = CreatePartner(4);
            var sales = CreateSales(4, 350000);
            var result = partner.GetDiscount(sales);
            Assert.AreEqual(15, result);
        }

        [TestMethod]
        public void GetDiscount_Returns_0_When_No_Sales()
        {
            var partner = CreatePartner(5);
            var sales = new List<ProductSale>();
            var result = partner.GetDiscount(sales);
            Assert.AreEqual(0, result);
        }
    }
}

using Microsoft.VisualStudio.TestTools.UnitTesting;
using ClassLibrary.Classes;

namespace TestSample
```

```
{
    [TestClass]
    public class ProductionTests
    {
        [TestMethod]
        public void CreateProduction_ShouldSetPropertiesCorrectly()
        {
            // Arrange
            var production = new Production
            {
                Id = 1,
                Article = "A123",
                Name = "Sample Product",
                Description = "Description of product",
                Photo = "photo_link.jpg",
                MinPartnerPrice = 99.99,
                Length = 50,
                Width = 20,
                Height = 10,
                NetWeight = 2.5,
                GrossWeight = 3.0,
                QualityCertificate = "QC12345",
                StandardNumber = "Std987",
                CostPrice = 70.00
            };

            // Act
            // Можно тут проверить свойства через Assert

            // Assert
            Assert.AreEqual(1, production.Id);
            Assert.AreEqual("A123", production.Article);
            Assert.AreEqual("Sample Product", production.Name);
            Assert.AreEqual("Description of product", production.Description);
            Assert.AreEqual("photo_link.jpg", production.Photo);
            Assert.AreEqual(99.99, production.MinPartnerPrice);
            Assert.AreEqual(50, production.Length);
            Assert.AreEqual(20, production.Width);
            Assert.AreEqual(10, production.Height);
            Assert.AreEqual(2.5, production.NetWeight);
            Assert.AreEqual(3.0, production.GrossWeight);
            Assert.AreEqual("QC12345", production.QualityCertificate);
            Assert.AreEqual("Std987", production.StandardNumber);
            Assert.AreEqual(70.00, production.CostPrice);
        }

        [TestMethod]
        public void
Production_DefaultConstructor_ShouldInitializePropertiesWithDefaults()
        {
            // Arrange & Act
            var production = new Production();

            // Assert
            Assert.AreEqual(0, production.Id);
            Assert.IsNull(production.Article);
            Assert.IsNull(production.Name);
            Assert.IsNull(production.Description);
            Assert.IsNull(production.Photo);
            Assert.AreEqual(0, production.MinPartnerPrice);
            Assert.IsNull(production.Length);
            Assert.IsNull(production.Width);
            Assert.IsNull(production.Height);
            Assert.IsNull(production.NetWeight);
            Assert.IsNull(production.GrossWeight);
```

```
            Assert.IsNull(production.QualityCertificate);
            Assert.IsNull(production.StandardNumber);
            Assert.IsNull(production.CostPrice);
        }

        // Дополнительные тесты можно добавить по необходимости, например,
        // проверки на валидацию данных, сериализацию и т.д.
    }
}

using Microsoft.VisualStudio.TestTools.UnitTesting;
using System;
using ClassLibrary.Classes; // импортируйте пространство имён вашего класса

namespace TestSample
{
    [TestClass]
    public class ProductSaleTests
    {
        [TestMethod]
        public void CreateProductSale_ShouldInitializeProperties()
        {
            // Arrange
            var saleId = 1;
            var partnerId = 100;
            var productionId = 200;
            var count = 10.5;
            var saleDate = DateOnly.FromDateTime(DateTime.Now);
            double? totalPrice = 150.75;

            // Act
            var productSale = new ProductSale
            {
                Id = saleId,
                PartnerId = partnerId,
                ProductionId = productionId,
                Count = count,
                SaleDate = saleDate,
                TotalPrice = totalPrice
            };

            // Assert
            Assert.AreEqual(saleId, productSale.Id);
            Assert.AreEqual(partnerId, productSale.PartnerId);
            Assert.AreEqual(productionId, productSale.ProductionId);
            Assert.AreEqual(count, productSale.Count);
            Assert.AreEqual(saleDate, productSale.SaleDate);
            Assert.AreEqual(totalPrice, productSale.TotalPrice);
        }

        [TestMethod]
        public void ProductSale_TotalPrice_CanBeNull()
        {
            // Arrange
            var sale = new ProductSale { TotalPrice = null };

            // Act & Assert
            Assert.IsNull(sale.TotalPrice);
        }
    }
}
}
```

Результаты тестирования



Заключение

Используя MS Visual Studio, технологии WPF и EntityFramework Core создали приложение для работы с партнерами компании. Сервер приложений реализован посредством создания WPF-приложения. Работа с данными в Excel- или BIN- файле реализована через динамическую библиотеку классов, написанного с использованием технологий Microsoft.Win32 и OfficeOpenXml.

Написанная программа работает, все основные (добавление, изменение, удаление, просмотр) и дополнительные (сортировка, фильтрация, поиск, сохранение и загрузка из файла) операции для работы с данными реализованы. Проведено тестирование, ошибки не выявлены.